

# Killer Sudoku

## Architecture Documentation

*nach arc42 v8.2*

|         |                                              |
|---------|----------------------------------------------|
| Project | Killer Sudoku (Variant: Cage Sums)           |
| Context | Skills Battle 2026 — Application Development |
| Author  | Tim Fischer                                  |
| Status  | Final                                        |
| Date    | 2026-05-27                                   |



# Inhaltsverzeichnis

**Architektur-Dokumentation** — Killer Sudoku · Skills Battle 2026 Standard: arc42 v8.2 · 12 Kapitel · Status: Final Author: Tim Fischer · Stand: 2026-05-27

## Kapitel

| #  | Kapitel                         |
|----|---------------------------------|
| 1  | Einführung und Ziele            |
| 2  | Randbedingungen                 |
| 3  | Kontextabgrenzung               |
| 4  | Lösungsstrategie                |
| 5  | Bausteinsicht                   |
| 6  | Laufzeitsicht                   |
| 7  | Verteilungssicht                |
| 8  | Querschnittliche Konzepte       |
| 9  | Architekturentscheidungen       |
| 10 | Qualitätsanforderungen          |
| 11 | Risiken und Technische Schulden |
| 12 | Glossar                         |

## Konventionen in diesem Dokument

| Konvention               | Bedeutung                                                                                  |
|--------------------------|--------------------------------------------------------------------------------------------|
| <b>Querverweise</b>      | "Kapitel X" verlinkt auf den entsprechenden Abschnitt in diesem Dokument                   |
| <b>Mermaid-Diagramme</b> | als SVG eingebettet (Klassen-, Sequenz-, ER-, Flow-Diagramme)                              |
| <b>Zitate</b>            | wörtliche Auszüge aus der Aufgabenstellung sind in Anführungszeichen gesetzt               |
| <b>ADR-Nummern</b>       | Kapitel 9 nummeriert Entscheidungen als ADR-001 bis ADR-014                                |
| <b>Identifizier</b>      | UCxx = Use Case · Vxx = Validation Rule · ACxx.y = Acceptance Criterion · Txxx = Test Case |



# 1 Einführung und Ziele

---

arc42 v8.2 · Skills Battle 2026 — Application Development — Killer Sudoku Quelldokument  
(autoritativ): **Aufgabenstellung**

---

## §1.1 Aufgabenstellung

Die Anwendung implementiert eine **Killer-Sudoku-Variante** als Web-Applikation. Anstelle des klassischen Sudoku werden zusätzlich **Cage-Summen** (Summen über mehrere Zellen) als Constraints vorgegeben. Die App muss Plan, Implementation und Tests gemäss den im Wettbewerbs-README spezifizierten Use Cases liefern.

### Spielregeln (wörtlich aus README §1)

Die Lösung eines Killer-Sudoku-Puzzles muss vier Bedingungen erfüllen (README §1, "the following conditions are met"):

"Each row, column, and nonet contains each number **exactly once**." "The sum of all numbers in a cage **must match** the small number printed in its corner." "**No** number appears more than once in a cage." "The solution **must be unique**."

Diese vier Regeln sind in der Anwendung als Validierungslogik abzubilden (siehe **V09**, **V10**) und werden in Use Case **UC09** sowie im Solver **UC11** geprüft.

### Funktionaler Scope

Die Anwendung deckt die im README §2.1 vorgegebenen Use Cases **UC01–UC11** sowie drei selbst gewählte zusätzliche Use Cases **UC12–UC14** ab (gemäss README §2.3, "Think of 3 additional use cases"):

- **UC01–UC11** — Read Rules, Register, Login, Enter Puzzle, Save Puzzle (Solvability-Check), Solve Puzzle, Hint, Highscore, Check Solution, Save Result, Auto Solve
- **UC12** — Browse / Filter Puzzles
- **UC13** — Pause & Resume Game
- **UC14** — Pencil Marks

Eine vollständige Übersicht inkl. Acceptance Criteria findet sich in **Use-Cases-Dokument**. Die Service-/Screen-/DB-Zuordnung ist in **Funktionalitäts-Matrix** tabelliert.

## Design-Freiheit (README §1.1)

Das README erlaubt explizite Designfreiheit: "You are free to design the application. Your task is to create mockups." Die Mockups sind in **Mockup-Briefings** als Figma-Briefings dokumentiert (Screens S1–S7 + Layout).

## §1.2 Qualitätsziele

Die folgenden drei Top-Qualitätsziele leiten alle Architektur- und Implementierungsentscheidungen. Sie sind so gewählt, dass sie aus den **strikten Vorgaben des README** ableitbar und über Tests messbar sind.

| #  | Qualitätsziel                                            | Begründung (README-Bezug)                                                                                                                                                                                                             | Messbarkeit                                                                                                                                                                            |
|----|----------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Q1 | <b>Korrektheit des Solvers (Funktionale Korrektheit)</b> | README §1 fordert "The solution <b>must be unique.</b> " und §2.1 UC4 fordert "Solutions <b>must be calculated with an algorithm.</b> " Der Solver ist die zentrale Logik-Komponente — falsche Ergebnisse zerstören UC05, UC07, UC09. | Solver-Unit-Tests gegen die 2 README-Beispiele + Edge-Cases (unlösbar, mehrdeutig). Performance-Boundary < 2s (siehe <b>AC11.2</b> ).                                                  |
| Q2 | <b>Lösungs-Eindeutigkeit (Konsistenz beim Speichern)</b> | README §1 "The solution must be unique." + §2.1 UC5 "The puzzle <b>can only be saved if it is solvable.</b> " Multi-Solution-Puzzles werden abgelehnt — nicht nur Zero-Solution (siehe <b>V07</b> ).                                  | Integration-Test: Speichern eines mehrdeutigen Puzzles → kein DB-Row.                                                                                                                  |
| Q3 | <b>Test-Abdeckung &amp; Test-Plan-Termin</b>             | README §3.1 fordert pro UC mind. 1 positiver + 1 negativer + (wo möglich) 1 Boundary-Test. README §1.4: "submission of the planned test cases must take place <b>by 11:30 o'clock.</b> "                                              | Test-Plan vor 11:30 abgegeben; Unit-Tests in Test-Framework lt. README §3.2 (" <b>should be implemented with a test framework</b> "). Coverage-Ziel siehe <a href="#">Kapitel 10</a> . |

## Weitere relevante Qualitätsmerkmale (sekundär)

- **Sicherheit** — README §2.1 UC2: "a login **is required.**" → Authentifizierung, geschützte Routen, sichere Passwort-Speicherung (siehe **V03, V16**).
- **Bedienbarkeit** — README §1.1 fordert "appropriate navigation" + "controls for all functions" → vollständig adressiert in **Mockup-Briefings**.
- **Wartbarkeit** — Klare Trennung Service-Layer / DB-Layer / UI (siehe Service-Interfaces in **Funktionalitäts-Matrix**).

## §1.3 Stakeholder

| Rolle                                                                      | Erwartung                                                                                                                                     | Berührungspunkte                                                                   |
|----------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <b>Wettbewerbs-Prüfer</b><br>(Skills Battle Jury)                          | Konforme Submission gemäss README §1.4 — ZIP-File, Doku als PDF, Source-Code, Executables, DB-Script. Bewertung gegen UCs und Test-Protokoll. | <code>AppDev_Name_FirstName.zip</code><br>(siehe <a href="#">Kapitel 2 §2.2</a> ). |
| <b>Wettbewerber</b> (Tim Fischer, Implementierer)                          | Zeit-effiziente, testbare Architektur. Strikte Vermeidung von Scope-Creep. Reproduzierbares lokales Setup.                                    | Source-Code, Test-Suite, lokaler Build mit <code>dotnet</code> .                   |
| <b>End-Nutzer</b> (Spieler der App)                                        | Funktionierender Killer-Sudoku-Spielfluss: Account → Puzzle wählen → spielen → Hint → Lösung prüfen → Highscore. UI-Sprache deutsch.          | UI-Screens S1–S7 ( <b>Mockup-Briefings</b> ).                                      |
| <b>Puzzle-Ersteller</b><br>(User-Subgruppe, kreiert Puzzles via UC04/UC05) | Verlässliche Solvability-Prüfung, kein Datenverlust bei Reject.                                                                               | Screen S5 — EnterPuzzle ( <b>Mockup-Briefings</b> ).                               |

### Nicht-Stakeholder (bewusst ausgegrenzt)

Folgende Rollen sind im Scope **nicht** vorgesehen (Begründung: nicht in README spezifiziert, keine UC-Anker):

- Admin-Rolle / Moderation
- Externe Systeme (Social-Login, Payment, Analytics)
- Mehrspieler-/Live-Funktionen

## Verweise

- Use Cases UC01–UC14
- Validation Rules V01–V16
- **Funktionalitäts-Matrix**
- [Kapitel 2 — Randbedingungen](#)
- [Kapitel 3 — Kontextabgrenzung](#)

## 2 Randbedingungen

arc42 v8.2 · Killer Sudoku App Quelle für Pflicht-Vorgaben: **Aufgabenstellung**

Randbedingungen sind **nicht-verhandelbare** Vorgaben, die das Design und die Implementierung einschränken. Sie sind in technische, organisatorische und Konventions-Constraints gegliedert.

### §2.1 Technische Randbedingungen

#### Plattform & Stack

| Constraint          | Wert                                   | Quelle / Begründung                                                                                                                                                                        |
|---------------------|----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Runtime / Framework | .NET 10                                | Wettbewerbs-Stack-Festlegung Tim Fischer. Dokumentiert in <b>Use-Cases-Dokument</b> (Stack-Header).                                                                                        |
| UI-Technologie      | Blazor Server                          | Server-Side Rendering mit SignalR-Circuit; spec-konform für interaktive Web-Anwendung lt. README §1.1 (Designfreiheit).                                                                    |
| Datenbank           | MS-SQL Server Express                  | README §1.3 erlaubt "MySQL or MS-SQL server running on your machine." Entscheidung: MS-SQL Express. DB-Name strikt: <code>sudoku</code> (README §1.3: "create a database named 'sudoku'"). |
| ORM / Data-Access   | Entity Framework Core 10 (oder Dapper) | Wahl noch offen; siehe <b>ER-Modell</b> . Beeinflusst Service-Interfaces nicht.                                                                                                            |
| Authentication      | ASP.NET Core Identity (Cookie-basiert) | Aus README §2.1 UC2 ("a login <b>is required</b> ") abgeleitet; siehe <b>V03 + V16</b> .                                                                                                   |

#### Test-Frameworks

README §3.2 (wörtlich): "Running the test cases **should be implemented with a test framework** and is part of the submission (e.g. JUnit in Java)." Für .NET ergibt sich:



| Test-Typ        | Framework         | Zweck                                                                                                                                           |
|-----------------|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Unit-Tests      | xUnit             | Service-Layer, Solver, Validation-Logic (Q1, Q3 — siehe §1.2).                                                                                  |
| Component-Tests | bUnit             | Blazor-Komponenten (Grid, CageEditor, Toolbar).                                                                                                 |
| E2E / UI-Tests  | Playwright (.NET) | Smoke-Tests kritischer Flows. README §3.2 erlaubt "UI tests can be run <b>without</b> a test framework" — Playwright ist optionale Erweiterung. |

## Datenbank-Setup (README §1.3, wörtlich zitiert)

- **Database-Name:** "sudoku" (Pflicht-Name aus README §1.3)
- **Schema-Freiheit:** "You are free to build the database schema."
- **SQL-Script-Pflicht:** "Create for each table a SQL statement and save them in a script called **Datenbank-Skript**."
- **Constraints-Pflicht:** "Don't forget to create the primary and foreign key constraints."

Das Schema und die Constraints sind in **ER-Modell** modelliert; das SQL-Script wird in `db/sudoku.sql` ausgeliefert.

## Browser-Support

| Browser                         | Status          | Begründung                                              |
|---------------------------------|-----------------|---------------------------------------------------------|
| <b>Chrome</b> (current stable)  | Pflicht-Target  | E2E-Tests laufen primär in Chromium-Engine.             |
| <b>Firefox</b> (current stable) | Sekundär-Target | Manuelle Smoke-Verifikation.                            |
| Safari / Edge                   | Best-effort     | Nicht im README spezifiziert, keine Test-Verpflichtung. |

## Entwicklungsumgebung (IDE-Auswahl)

| IDE                                    | Status                                         |
|----------------------------------------|------------------------------------------------|
| <b>Visual Studio 2022/2026</b>         | Primär (Blazor-Server-Templates, MS-SQL-Tools) |
| <b>Visual Studio Code</b> + C# Dev Kit | Alternative                                    |
| <b>JetBrains Rider</b>                 | Alternative                                    |

Die Wahl ist intern; das Build- und Test-Setup muss in **allen drei** funktionieren (kein IDE-gebundenes Projekt-Setup).

**UNCLEAR:** Eine exakte minimale .NET-SDK-Version wird vom README nicht vorgegeben.  
Annahme: .NET 10 (LTS-Kandidat zum Wettbewerbszeitpunkt).

## §2.2 Organisatorische Randbedingungen

### Submission-Termine (README §1.4 + §3.1)

| Termin                               | Pflicht (wörtliches Zitat)                                                                            |
|--------------------------------------|-------------------------------------------------------------------------------------------------------|
| <b>12:00 Uhr</b><br>(Wettbewerbstag) | "submission of the planned test cases <b>must take place by 12 o'clock</b> "<br>(README §1.4 + §3.1). |
| <b>Ende Wettbewerbstag</b>           | Komplette Submission (ZIP) gemäss README §1.4.                                                        |

### Deliverables (README §1.4, wörtlich)

"All deliverables must be submitted in a zip file named `AppDev_Name_FirstName.zip` ." "Only the contents of the zip file will be considered for the evaluation."

Pflicht-Inhalte der ZIP-Datei:

| Deliverable                           | Format                                                                                                                                                       | README-Zitat                                                  |
|---------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| Dokumentation<br>inkl. Test-Protokoll | <b>PDF</b>                                                                                                                                                   | §2.6: "For the delivery, create a <b>PDF document</b> ."      |
| Executable Files                      | Build-Artefakte (.exe / <code>dotnet publish</code> Output) + <b>Container-Image-Tag</b> ( <code>ghcr.io/tim-fischer-zh/killer-sudoku:latest</code> )        | §1.4: "executable files"                                      |
| Source-Code                           | Vollständig, inkl. <code>Dockerfile</code> , <code>docker-entrypoint.sh</code> , <code>docker-compose.yml</code> , <code>.github/workflows/docker.yml</code> | §1.4: "the source code"                                       |
| DB-Script                             | <b>Datenbank-Skript</b> (im Container automatisch beim ersten Start applied)                                                                                 | §1.3: "save them in a script called <code>sudoku.sql</code> " |

### Container-Variante (zusätzlich zur Native-Variante)

Die Submission liefert ein **Single-Image-Container-Setup** als bevorzugten Startpfad (siehe [ADR-015](#) und [§7.5](#)). Der Prüfer kann zwischen drei Pfaden wählen:

1. `docker pull` + `docker run` — Image direkt von `ghcr.io` ( $\approx 1.5$  GB), App in unter 60 Sek startklar.
2. `docker compose up --build` — lokal aus dem Repo-Root bauen.
3. **Native** — MS-SQL Express + .NET 10 SDK lokal, **Datenbank-Skript** manuell deployen (Standard-Pfad, vollständig in [§7.2](#) / [§7.3](#) dokumentiert).

README §1.3 ("running on your machine") wird in allen drei Pfaden erfüllt — Container laufen auf der Maschine des Prüfers.

## Dokumentations-Inhalt (README §2.6, wörtlich)

Die Dokumentation muss folgende Sektionen enthalten:

"Mockup · Database diagram · Class diagram · Additionally chosen use cases · Validation rules · Test protocol"

Diese Punkte werden über die arc42-Kapitel und die in `docs/` referenzierten Spezialdokumente abgedeckt:

| README-Sektion                | Quelle im Repo                                            |
|-------------------------------|-----------------------------------------------------------|
| Mockup                        | <b>Mockup-Briefings</b> + <code>docs/mockups/*.png</code> |
| Database diagram              | <b>ER-Modell</b>                                          |
| Class diagram                 | <a href="#">Kapitel 5 — Bausteinsicht</a> (TBD)           |
| Additionally chosen use cases | <b>Use-Cases-Dokument</b> (UC12–UC14)                     |
| Validation rules              | <b>Validation-Regeln</b>                                  |
| Test protocol                 | <b>Test-Protokoll</b> (TBD)                               |

## Bewertungsmaterial-Beschränkung

README §1.4: "Only the contents of the zip file will be considered for the evaluation."

Konsequenz: Externe Repos, Cloud-Builds oder Online-Demos werden **nicht** in die Bewertung einfließen. Alles, was zur Bewertung beitragen soll, muss im ZIP enthalten sein.

## §2.3 Konventionen

### Sprach-Konvention

| Bereich                                                 | Sprache         | Begründung                                                                                                                                                  |
|---------------------------------------------------------|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>UI-Texte</b> (Buttons, Labels, Fehlermeldungen)      | <b>Deutsch</b>  | Implementierer + Zielgruppe in DACH-Region; konsistent mit Mockup-Briefings ( <b>Mockup-Briefings</b> ) und Validation-Texten ( <b>Validation-Regeln</b> ). |
| <b>Code-Identifizier</b> (Klassen, Methoden, Variablen) | <b>Englisch</b> | .NET-Konvention; siehe Service-Interfaces in <b>Funktionalitäts-Matrix</b> .                                                                                |
| <b>Architektur-Doku (arc42)</b>                         | <b>Deutsch</b>  | arc42-Standard im DACH-Kontext; README-Zitate bleiben original-englisch in "...".                                                                           |
| <b>DB-Schema-Namen</b>                                  | <b>Englisch</b> | Englische Tabellennamen ( <code>AppUser</code> , <code>Puzzle</code> , <code>Cage</code> , ...); siehe <b>ER-Modell</b> .                                   |

### Code-Style

- **C# Style:** Microsoft .NET Coding Conventions (PascalCase für Public, camelCase für lokal, `_camelCase` für private fields).
- **Nullable Reference Types:** aktiviert ( `<Nullable>enable</Nullable>` im csproj).
- **Async-Konvention:** I/O-bound Methoden mit `Async` -Suffix und `Task` -Return — sichtbar an Service-Interfaces (z.B. `IPuzzleService.SaveIfSolvableAsync` , siehe **Funktionalitäts-Matrix**).
- **Immutability bevorzugt** wo idiomatisch (DTOs als `record` ).

### Sicherheits-Konventionen (aus Validation-Regeln abgeleitet)

| Konvention                                                                           | Quelle                          |
|--------------------------------------------------------------------------------------|---------------------------------|
| <b>Login required</b> für alle geschützten Routen (Puzzles, Spielen, Highscore)      | README §2.1 UC2 + <b>V16</b>    |
| <b>Passwort wird NIE im Klartext gespeichert</b>                                     | <b>V03</b>                      |
| <b>Generische Login-Fehlermeldung</b> (kein User-Existenz-Hinweis)                   | <b>V03 + AC03.1</b>             |
| <b>CSRF-Token</b> auf allen POST/Form-Submissions (Blazor-Standard)                  | <b>V15</b>                      |
| <b>HTML-Encoding</b> auf User-Content ( <code>@</code> -Razor-Syntax)                | <b>V14</b>                      |
| <b>Defense in Depth:</b> kritische Validierungen auf Client + Server + DB-Constraint | Header <b>Validation-Regeln</b> |

### Verweise

- [Kapitel 1 — Einführung und Ziele](#)

- [Kapitel 3 — Kontextabgrenzung](#)
- Validation Rules
- **ER-Modell**
- **Funktionalitäts-Matrix**

## 3 Kontextabgrenzung

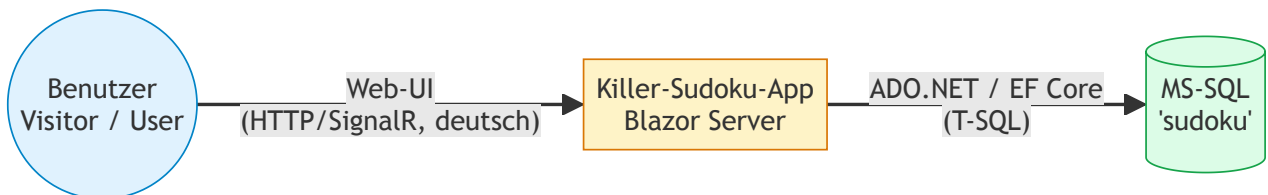
arc42 v8.2 · Killer Sudoku App Quelle (autoritativ): **Aufgabenstellung**

Die Kontextabgrenzung beschreibt die Systemgrenze: **welche Akteure** mit der Anwendung interagieren, **welche Nachbarsysteme** angesprochen werden, und über **welche Schnittstellen** die Kommunikation läuft. Sie unterscheidet fachlich (welche Akteure / welche Use Cases) und technisch (welche Protokolle / Komponenten).

### §3.1 Fachlicher Kontext

#### System-Grenze

Die Killer-Sudoku-App ist ein **lokal betriebenes Single-User-System** mit einer Web-UI. Es existieren **keine externen Fachsysteme** (kein Drittanbieter-Login, keine externen APIs, keine Bezahl-/Mailservices).



#### Akteure

| Akteur                                       | Beschreibung                                                               | Berührungspunkte (UC)                                            |
|----------------------------------------------|----------------------------------------------------------------------------|------------------------------------------------------------------|
| <b>Visitor</b>                               | Nicht angemeldeter Browser-Nutzer. Darf nur die öffentlichen Seiten sehen. | <b>UC01</b> (Rules), <b>UC02</b> (Register), <b>UC03</b> (Login) |
| <b>User</b>                                  | Angemeldeter Spieler. Darf Puzzles erstellen, spielen, Highscore sehen.    | <b>UC04–UC10</b> , <b>UC12–UC14</b>                              |
| <b>System</b> (intern, kein externer Akteur) | Triggert automatische Folge-UCs (z.B. UC10 nach erfolgreichem UC09).       | <b>UC10</b> , <b>UC11</b> (intern aufgerufen von UC05/UC07/UC09) |

#### Keine externen Akteure / Nachbar-Systeme

Folgendes ist **bewusst nicht** Teil des Kontexts (siehe [Kapitel 2](#)):

- Kein Identity-Provider (Google/Microsoft/Apple-SSO)
- Kein Email-Service (kein Email-Verifikations- oder Password-Reset-Flow im Scope)

- Keine externen Highscore-/Leaderboard-APIs
- Kein Cloud-Backend — Anwendung läuft lokal mit lokaler DB

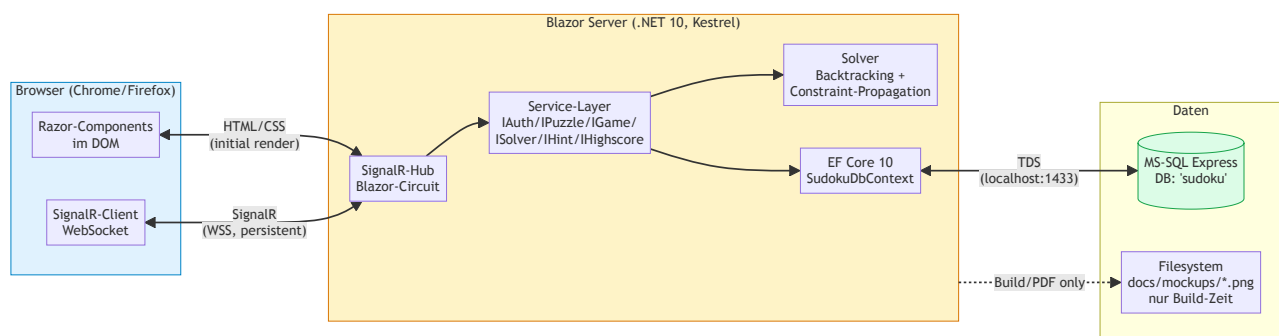
README §1.3 (wörtlich): "create a database named 'sudoku' on MySQL or MS-SQL server **running on your machine.**" → klare Lokal-Vorgabe.

## Eingaben / Ausgaben über die System-Grenze

| Richtung   | Daten                                                                                                                                                                                                                                  | Kanal                                                           |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| User → App | Login-Credentials, Puzzle-Definition (Cages + Summen), Zelleneingaben (1–9), Hint-/Check-/Pause-Aktionen                                                                                                                               | HTTP-POST (initial) + SignalR-WebSocket (Blazor-Server-Circuit) |
| App → User | Gerendertes 9×9-Grid, Cage-Borders + Summen, Timer, Hint-Ergebnis, Lösungs-Validierungs-Status, Highscore-Liste, Fehlermeldungen (deutsch)                                                                                             | HTML/CSS (initial) + SignalR-Patches (UI-Updates)               |
| App → DB   | INSERT/UPDATE/SELECT auf <code>AppUser</code> , <code>Puzzle</code> , <code>Cage</code> , <code>CageCell</code> , <code>Game</code> , <code>GameCell</code> , <code>PencilMark</code> , <code>HintLog</code> (siehe <b>ER-Modell</b> ) | EF Core über ADO.NET / TDS-Protokoll                            |
| DB → App   | Abfrage-Resultate, generierte Primary Keys, View-Daten ( <code>vw_Highscore</code> )                                                                                                                                                   | TDS                                                             |

## §3.2 Technischer Kontext

### Komponenten & Protokolle



## Schnittstellen-Details

| Schnittstelle                        | Protokoll                                     | Format                    | Notiz                                                                   |
|--------------------------------------|-----------------------------------------------|---------------------------|-------------------------------------------------------------------------|
| Browser ↔ Server (Initial-Page-Load) | HTTPS                                         | HTML                      | Erste Request liefert Razor-generiertes HTML inkl. Blazor-Bootstrapper. |
| Browser ↔ Server (UI-Interaktion)    | SignalR über WebSocket (WSS)                  | binär (MessagePack)       | Persistente Verbindung pro Blazor-Circuit. UI-State serverseitig.       |
| Server ↔ DB                          | TDS via <code>Microsoft.Data.SqlClient</code> | T-SQL                     | Local-Loopback ( <code>localhost:1433</code> oder Named Pipe).          |
| Server ↔ Filesystem                  | nur Build-Zeit                                | PNG (Mockups), PDF (Doku) | Keine Runtime-File-IO ausser Logs.                                      |

## Blazor-Server-Circuit (zentrales technisches Konzept)

- **State serverseitig:** Game-State, aktuelle Zell-Auswahl, Pencil-Marks-Status leben in C#-Components auf dem Server.
- **UI-Updates push-basiert:** Server pusht Diff-Patches an den Client über SignalR.
- **Skalierung:** Eine Circuit pro Browser-Tab. Bei Disconnect → State kann via Reconnect wiederhergestellt werden (Standard-Blazor-Mechanismus; siehe auch **UC13** für explizite Pause/Resume-Semantik).

## Lokales Setup (Single-Machine-Deployment)

Aus README §1.3 ("running on your machine") folgt: Die App und die DB laufen **auf dem gleichen Host**. Es gibt keinen Multi-Tier-Deployment-Plan. Details siehe spätere Kapitel:

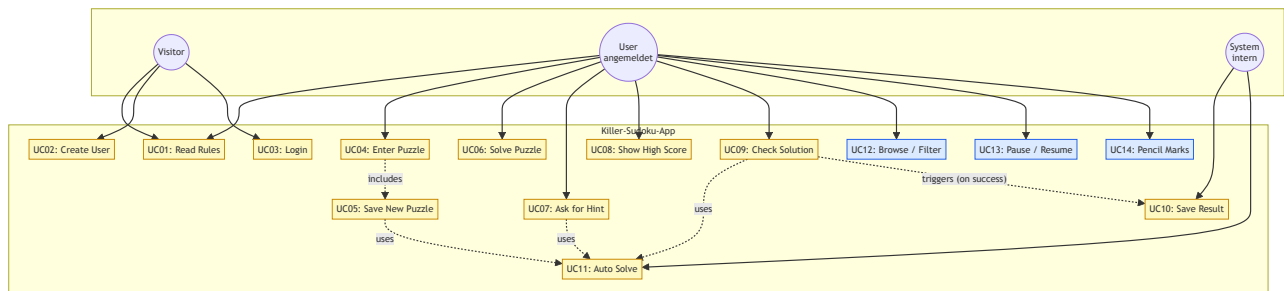
- [Kapitel 5 — Bausteinsicht](#) (TBD): innere Struktur Service-Layer
- [Kapitel 7 — Verteilungssicht](#) (TBD): lokales Deployment-Diagramm

## §3.3 Use-Case-Übersicht

Vollständige UC-Spezifikation: siehe **Use-Cases-Dokument**. Diese Sektion liefert nur die Aktor-Sicht und das Diagramm als Kontext-Anker.



## Use-Case-Diagramm



**Legende:** Gelb = README §2.1 (UC01–UC11) · Blau = selbst gewählte Zusatz-UCs §2.3 (UC12–UC14).

## Kurz-Mapping (Detail in Use-Cases-Dokument)

| UC          | Kurzbeschreibung                                        | Akteur         | Auth     |
|-------------|---------------------------------------------------------|----------------|----------|
| <b>UC01</b> | Regeln + Beispiel auf Startseite                        | Visitor / User | public   |
| <b>UC02</b> | Account erstellen                                       | Visitor        | public   |
| <b>UC03</b> | Anmelden                                                | Visitor        | public   |
| <b>UC04</b> | Puzzle manuell anlegen (Cages + Summen, Difficulty 1–3) | User           | required |
| <b>UC05</b> | Puzzle speichern — nur wenn solvable + eindeutig        | User           | required |
| <b>UC06</b> | Puzzle spielen                                          | User           | required |
| <b>UC07</b> | Hint anfordern                                          | User           | required |
| <b>UC08</b> | Highscore-Liste                                         | User           | required |
| <b>UC09</b> | Lösung prüfen (Sum-Check 405 → Vollvalidierung)         | User           | required |
| <b>UC10</b> | Game-Resultat speichern (Time, Hints, Score)            | System         | required |
| <b>UC11</b> | Solver (intern für UC05/UC07/UC09)                      | System         | n/a      |
| <b>UC12</b> | Puzzle-Liste mit Filter/Sort                            | User           | required |
| <b>UC13</b> | Spiel pausieren und fortsetzen                          | User           | required |
| <b>UC14</b> | Candidate-Notes pro Zelle                               | User           | required |

Use-Case-Cluster (zur Test-Planung)

| Cluster           | UCs                                | Test-Fokus                                  |
|-------------------|------------------------------------|---------------------------------------------|
| Authentifizierung | UC02, UC03                         | Sicherheit (V01–V04, V16)                   |
| Puzzle-Lifecycle  | UC04, UC05, UC11                   | Solver-Korrektheit (Q1), Eindeutigkeit (Q2) |
| Spiel-Flow        | UC06, UC07, UC09, UC10, UC13, UC14 | Game-State-Konsistenz (V12)                 |
| Diskover & Stats  | UC01, UC08, UC12                   | Read-Pfade, Sortierung/Pagination           |

Verweise

- [Kapitel 1 — Einführung und Ziele](#)
- [Kapitel 2 — Randbedingungen](#)
- Use Cases UC01–UC14 (Detail)
- Validation Rules
- **Funktionalitäts-Matrix**
- **ER-Modell**
- **Mockup-Briefings**

## 4 Lösungsstrategie

---

arc42 v8.2 · Kapitel 4 · Killer Sudoku Bezug: [00-README.md](#) (Index) · **Use-Cases-Dokument** · **Funktionalitäts-Matrix** · **Validation-Regeln** · **ER-Modell** · **Aufgabenstellung**

Dieses Kapitel fasst die fundamentalen Entscheidungen zusammen, die alle nachfolgenden Kapitel tragen. Die Aufgabenstellung gibt **kein** spezifisches Framework vor (siehe §1.3 "free to build the database schema", §1 "your task is to plan, implement, and test"). Damit sind Technologie-Wahl und Top-Level-Zerlegung explizit Architektur-Entscheidungen — sie sind hier dokumentiert.

---

### 4.1 Technologie-Entscheidungen

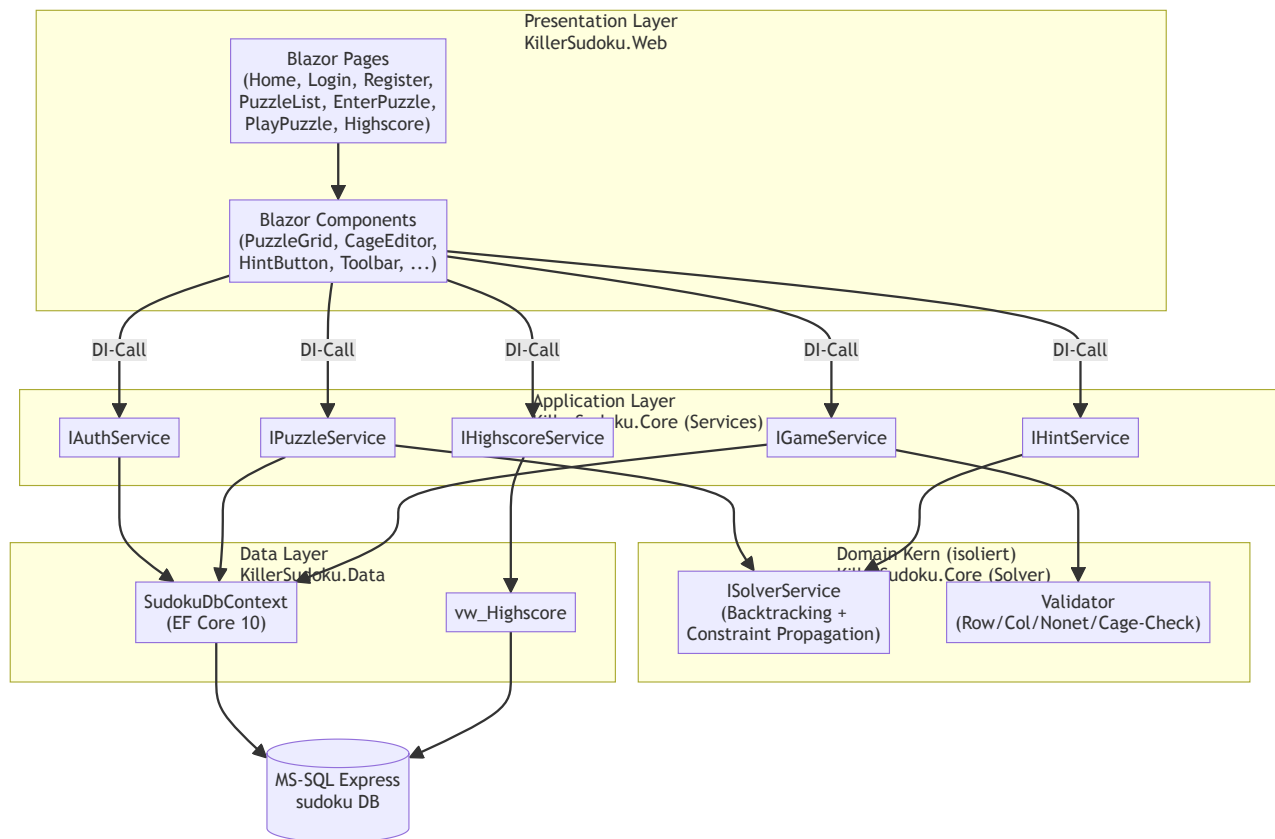
Die folgende Tabelle ordnet jede zentrale Anforderung aus dem Aufgaben-README einer technischen Lösung zu. Vollständige Begründung in [Kapitel 9 — Architekturentscheidungen](#) (TBD).

| Anforderung<br>(Quelle)                               | Gewählte Lösung                                                                                                                       | Begründung                                                                                                                                                                                             |
|-------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| "login is required"<br>(UC02/UC03)                    | <b>ASP.NET Core Identity</b> (Cookie-Auth)                                                                                            | Built-in für .NET 10, PBKDF2-Hashing, CSRF/XSRF-Token frei Haus, deckt <b>V03/V15</b> Defense-in-Depth                                                                                                 |
| "stored in the database"<br>(UC04)                    | <b>Microsoft SQL Server Express</b> + Entity Framework Core 10                                                                        | README erlaubt explizit MS-SQL oder MySQL; MS-SQL Express ist lokal lizenzkostenfrei, native T-SQL-Constraints (CHECK/UNIQUE/Filtered Index/Trigger) tragen DB-Layer-Verteidigung aus <b>ER-Modell</b> |
| "must be calculated with an algorithm"<br>(UC04/UC11) | <b>Reiner C#-Solver</b> ( <code>ISolverService</code> ) als isolierter Domain-Kern ohne Framework-Abhängigkeit                        | Pure Logik → Unit-testbar ohne DB/UI; Backtracking + Constraint-Propagation; siehe <b>Funktionalitäts-Matrix</b> Service-Block                                                                         |
| "solution must be unique" (§1)                        | Solver bricht nach dem <b>zweiten</b> gefundenen Pfad ab → <code>Solutions ∈ {0,1,2}</code> ( <b>V07</b> )                            | Performance-Begrenzung (AC11.2: <2 s); 2 ist Marker für "≥2"                                                                                                                                           |
| Interaktives Grid<br>(UC04, UC06, UC07, UC14)         | <b>Blazor Server</b> (Razor Components + SignalR)                                                                                     | Server-rendered: Solver/Validierung läuft serverseitig (kein clientseitiges Cheaten möglich), <code>[Authorize]</code> -Attribut für Routen, kein zweites SPA-Frontend-Projekt nötig                   |
| "test cases ... unit tests ... test framework" (§3)   | <b>xUnit + bUnit</b> (Component-Tests) + <b>EF Core InMemory / Testcontainers</b>                                                     | Standard für .NET; bUnit erlaubt Blazor-Komponenten-Tests ohne Browser; siehe <b>Test-Protokoll</b>                                                                                                    |
| Validation-Defense (V01–V16)                          | <b>3 Layer</b> : Client (Blazor <code>EditForm</code> /Annotations) → Server ( <code>IXxxService</code> ) → DB (CHECK/UNIQUE/Trigger) | "Defense in Depth" aus <b>Validation-Regeln</b> Header — kritische Regeln auf mind. 2 Layern                                                                                                           |
| Sum-Check "405" (§2.3)                                | <b>Fast-Fail in</b> <code>IGameService.CheckSolutionAsync</code> vor Voll-Algorithmus ( <b>V09</b> )                                  | README §2.3: "Use the value for a simple validation before checking the solution with an algorithm" — wörtlich umgesetzt                                                                               |

**Hinweis:** Die Solver-Implementierung wird über das Interface `ISolverService` aus **Funktionalitäts-Matrix** entkoppelt — der konkrete Backtracker ist eine austauschbare Detail-Entscheidung, kein Architektur-Pfeiler.

## 4.2 Top-Level-Zerlegung

Das System wird in eine **klassische 3-Layer-Architektur** zerlegt, ergänzt um einen **isolierten Domain-Kern** für die rechenintensive Solver-Logik. Diese Zerlegung erscheint im Detail in [Kapitel 5 — Bausteinsicht](#).



### Schichten-Verantwortung:

| Schicht                                              | Verantwortung                                                                    | Abhängigkeit nach                              |
|------------------------------------------------------|----------------------------------------------------------------------------------|------------------------------------------------|
| <b>Presentation</b><br>( KillerSudoku.Web )          | UI-Rendering, Routing, User-Interaktion, Input-Binding, Display-State            | nur <b>Application</b> (über Interfaces)       |
| <b>Application</b><br>( KillerSudoku.Core Services ) | Use-Case-Orchestrierung, Transaktions-Grenzen, Autorisierung, Aufruf des Solvers | <b>Domain + Data</b>                           |
| <b>Domain-Kern</b><br>( KillerSudoku.Core Solver )   | Reine Algorithmik (Backtracking, Constraint-Propagation, Sum-Check)              | <b>keine</b> (pure C# — kein EF, kein ASP.NET) |
| <b>Data</b> ( KillerSudoku.Data )                    | EF-Core-Mapping, Migrations, Query-Performance, View-Zugriff                     | DB-Treiber                                     |

**Der Solver als isolierter Domain-Kern** ist die wichtigste Designtrennung: Er ist die kritische Geschäftslogik (UC11 wird von UC05, UC07, UC09 verwendet — siehe Abhängigkeits-Graph in **Funktionalitäts-Matrix** §Kritische Abhängigkeiten) und muss separat per Unit-Test verifiziert werden. Pure C# ohne Framework-Bindung erlaubt Tests in Millisekunden statt Sekunden.

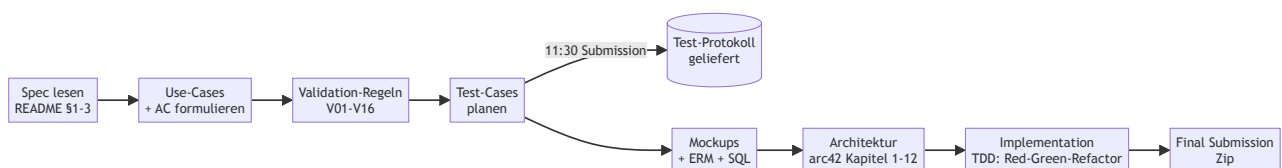
## 4.3 Erreichen der Qualitätsanforderungen

Die wichtigsten Qualitätsziele werden durch konkrete Patterns adressiert. Vollständige Liste in [Kapitel 10](#) (TBD); hier nur die strategie-prägenden Mappings.

| Qualitätsziel                                                          | Pattern / Mechanismus                                                                                                                                                                                                                                                   | Bezug                        |
|------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------|
| <b>Korrektheit</b> — Sudoku- und Killer-Regeln werden niemals verletzt | (a) DB-Constraints als letzte Verteidigung ( <b>V05</b> , <b>V06</b> , <b>V08</b> ). (b) Server-seitige Re-Validierung in <code>IGameService.CheckSolutionAsync</code> (UC09). (c) Solver muss "solution must be unique" garantieren ( <b>V07</b> ).                    | README §1 Cage-Regeln        |
| <b>Performance</b> — Solver < 2 s, UI-Reaktion < 200 ms                | (a) Sum-Check 405 als Fast-Fail vor Algorithmus (AC09.1). (b) Solver bricht bei der 2. Lösung ab (AC11.2). (c) Paginierte Puzzle-Listen (AC12.2). (d) <code>vw_Highscore</code> als pre-joined VIEW (siehe <b>ER-Modell</b> ).                                          | UC11 AC11.2                  |
| <b>Testbarkeit</b> — 80 %+ Coverage, TDD-fähig                         | (a) Service-Interfaces ( <code>I*Service</code> ) erlauben Mock-Substitution. (b) Solver als pure Funktion testbar ohne DB. (c) bUnit für Component-Tests, EF InMemory für Integration.                                                                                 | <b>Test-Protokoll</b>        |
| <b>Sicherheit</b> — kein Klartext-Passwort, kein XSS, kein CSRF        | (a) ASP.NET Identity PBKDF2 ( <b>V03</b> ). (b) Razor <code>@</code> -Encoding ( <b>V14</b> ). (c) Antiforgery-Token in EditForm ( <b>V15</b> ). (d) <code>[Authorize]</code> + UserId-Ownership ( <b>V16</b> ).                                                        | UC02/UC10/UC13               |
| <b>Wartbarkeit</b> — kleine, fokussierte Files                         | "Many small files > few large files" aus globalen Code-Standards: 200–400 Zeilen pro Komponente, Service oder Repo. Komponenten-Tree spiegelt Domain-Begriffe.                                                                                                          | Coding-Style-Regel           |
| <b>Datenintegrität</b> — keine inkonsistenten Game-States              | (a) Filtered Unique Index <code>UX_Game_ActiveOnly</code> (max 1 aktives Game pro User-Puzzle). (b) Trigger <code>trg_CageCell_UniquePerPuzzle</code> (Cell-Cage-Eindeutigkeit). (c) Transaktion in <code>SaveIfSolvableAsync</code> (Puzzle + Cage + CageCell atomar). | <b>ER-Modell</b> Constraints |

## 4.4 Organisatorische Strategie

Die Aufgabenstellung schreibt einen **klaren zeitlichen Ablauf** vor (§3.1: "Submission der geplanten Test-Cases bis 12 Uhr", §1.4 "Doku + Source + DB-Script"). Das diktiert die Reihenfolge der Artefakte:



## Tragende organisatorische Prinzipien:

1. **Spec → Tests → Code** (TDD): Jeder Test-Case in **Test-Protokoll** verlinkt auf eine `AC*`-ID; jede AC referenziert wörtlich den README-Text via "Source"-Zeile.
2. **Solver zuerst:** Da `ISolverService` Pflicht-Dependency für UC05, UC07, UC09 ist (siehe **Funktionalitäts-Matrix** §Kritische Abhängigkeiten), wird er als erstes implementiert und mit Beispielen aus den 2 README-Examples (§1.2) sowie Edge-Cases (unsolvable, multi-solution) abgesichert.
3. **Validation auf 3 Layern:** Jede strikte Constraint wird auf Client + Server + DB redundant gesichert. Dies macht Tests gegen einzelne Layer wertvoll, statt Doppel-Tests.
4. **Mockup vor Implementierung:** README §1.1 verlangt Mockups als Doku-Bestandteil — sie werden separat erstellt (**Mockup-Briefings**) und sind die UI-Spezifikation, gegen die Blazor-Components gebaut werden.
5. **arc42 referenziert, dupliziert nicht:** Die Quell-Dokumente (**Use-Cases-Dokument**, **Funktionalitäts-Matrix**, **Validation-Regeln**, **ER-Modell**) sind die Single Source of Truth — arc42-Kapitel verlinken sie statt sie zu kopieren. So gibt es keine driftenden Mehrfach-Quellen.

### Nächste Kapitel:

- [Kapitel 5 — Bausteinsicht](#): Detail der drei Layer mit Klassen, Schnittstellen, Verantwortungen.
- [Kapitel 6 — Laufzeitsicht](#): Vier Schlüssel-Szenarien (Puzzle-Save, Hint, Check, Pause/Resume) als Sequenz-Diagramme.

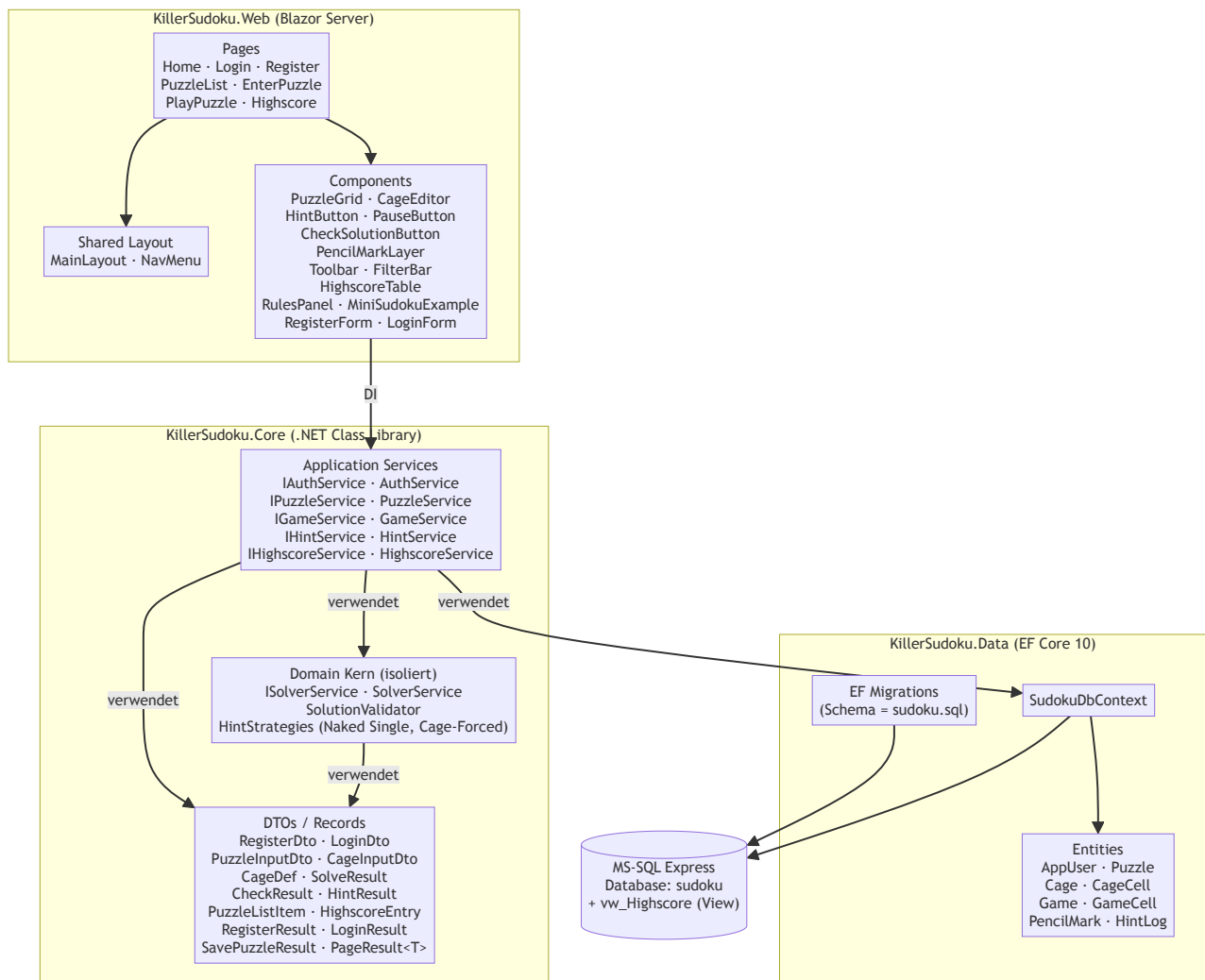
# 5 Bausteinsicht

arc42 v8.2 · Kapitel 5 · Killer Sudoku Bezug: [Kapitel 4](#) · **Funktionalitäts-Matrix** (autoritativ für Service-Signaturen) · **ER-Modell** · **Datenbank-Skript**

Dieses Kapitel zerlegt das System statisch in seine Bausteine. Die Service-Signaturen werden **nicht** dupliziert — sie stehen in **Funktionalitäts-Matrix** §Service-Interfaces. Hier wird die Architektur-Sicht beschrieben: Zweck, Schnittstellen, enthaltene Klassen, Abhängigkeiten.

## 5.1 Whitebox Gesamtsystem

Die Gesamtanwendung besteht aus drei Projekten/Assemblies (vorgesehene Aufteilung in der .NET-Solution):



**Projekt-Verantwortlichkeiten kurz:**



- `KillerSudoku.Web` — UI-Schicht; rein präsentational; keine Geschäftslogik; ruft Services per DI auf.
- `KillerSudoku.Core` — Application Services + isolierter Domain-Kern (Solver/Validator) + DTOs; **keine** EF-Core-Abhängigkeit im Domain-Kern.
- `KillerSudoku.Data` — Persistenz; EF-Core-DbContext, Entity-Mappings, Migrations. Wird von Application-Services verwendet, aber nicht von der UI direkt.

## 5.2 Baustein-Tabellen (Whitebox je Layer)

Für jeden Baustein: Zweck · Schnittstellen (eingehend/ausgehend) · enthaltene Klassen/Komponenten. Service-Signaturen siehe **Funktionalitäts-Matrix**.

### 5.2.1 KillerSudoku.Web — Presentation Layer

| Aspekt                          | Details                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Zweck</b>                    | Blazor-Server-Anwendung; rendert UI, leitet User-Aktionen an Application-Services weiter, zeigt Ergebnisse.                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <b>Eingehende Schnittstelle</b> | HTTP-Requests vom Browser (Routes aus <b>Funktionalitäts-Matrix</b> §Screen-Inventar S1–S8); SignalR-Verbindung für interaktive Updates.                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Ausgehende Schnittstelle</b> | Application-Services via DI: <code>IAuthService</code> , <code>IPuzzleService</code> , <code>IGameService</code> , <code>IHintService</code> , <code>IHighscoreService</code> .                                                                                                                                                                                                                                                                                                                                                                     |
| <b>Pages ( @page )</b>          | <code>Home.razor</code> (S1, /) · <code>Register.razor</code> (S2, /register) · <code>Login.razor</code> (S3, /login) · <code>PuzzleList.razor</code> (S4, /puzzles) · <code>EnterPuzzle.razor</code> (S5, /puzzles/new) · <code>PlayPuzzle.razor</code> (S6, /puzzles/{id}/play) · <code>Highscore.razor</code> (S7, /highscore).                                                                                                                                                                                                                  |
| <b>Shared Components</b>        | <code>MainLayout.razor</code> · <code>NavMenu.razor</code> · <code>RulesPanel.razor</code> · <code>MiniSudokuExample.razor</code> · <code>RegisterForm.razor</code> · <code>LoginForm.razor</code> · <code>PuzzleGrid.razor</code> · <code>CageEditor.razor</code> · <code>HintButton.razor</code> · <code>CheckSolutionButton.razor</code> · <code>PauseButton.razor</code> · <code>PencilMarkLayer.razor</code> · <code>Toolbar.razor</code> · <code>FilterBar.razor</code> · <code>PuzzleCard.razor</code> · <code>HighscoreTable.razor</code> . |
| <b>Querschnitt</b>              | <code>[Authorize]</code> auf allen Pages außer S1/S2/S3 ( <b>V16</b> ); <code>EditForm</code> mit Antiforgery ( <b>V15</b> ); Razor-@-Encoding ( <b>V14</b> ).                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Test-Strategie</b>           | bUnit-Component-Tests (Rendering, Event-Handler), E2E ohne Framework gemäß README §3.2.                                                                                                                                                                                                                                                                                                                                                                                                                                                             |

### 5.2.2 KillerSudoku.Core — Application Services

| Aspekt                          | Details                                                                                                                                                                                                                  |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Zweck</b>                    | Use-Case-Orchestrierung; konvertiert UI-Eingaben in DB-Operationen + Domain-Aufrufe; setzt Transaktions- und Autorisierungs-Grenzen.                                                                                     |
| <b>Eingehende Schnittstelle</b> | Service-Interfaces <code>IAuthService</code> , <code>IPuzzleService</code> , <code>IGameService</code> , <code>IHintService</code> , <code>IHighscoreService</code> — komplette Signaturen siehe Funktionalitäts-Matrix. |
| <b>Ausgehende Schnittstelle</b> | <code>ISolverService</code> (Domain) + <code>SudokuDbContext</code> (Data).                                                                                                                                              |
| <b>Enthaltene Services</b>      | <code>AuthService</code> (UC02/UC03) · <code>PuzzleService</code> (UC04/UC05/UC12) · <code>GameService</code> (UC06/UC09/UC10/UC13/UC14) · <code>HintService</code> (UC07) · <code>HighscoreService</code> (UC08).       |
| <b>UC-Mapping</b>               | Vollständig in <b>Funktionalitäts-Matrix</b> §Matrix-Tabelle.                                                                                                                                                            |
| <b>Test-Strategie</b>           | Unit-Tests pro Service mit gemocktem <code>ISolverService</code> + <code>SudokuDbContext</code> (EF InMemory); Integration-Tests gegen echte MS-SQL-Test-DB.                                                             |

### 5.2.3 KillerSudoku.Core — Domain-Kern (Solver/Validator)

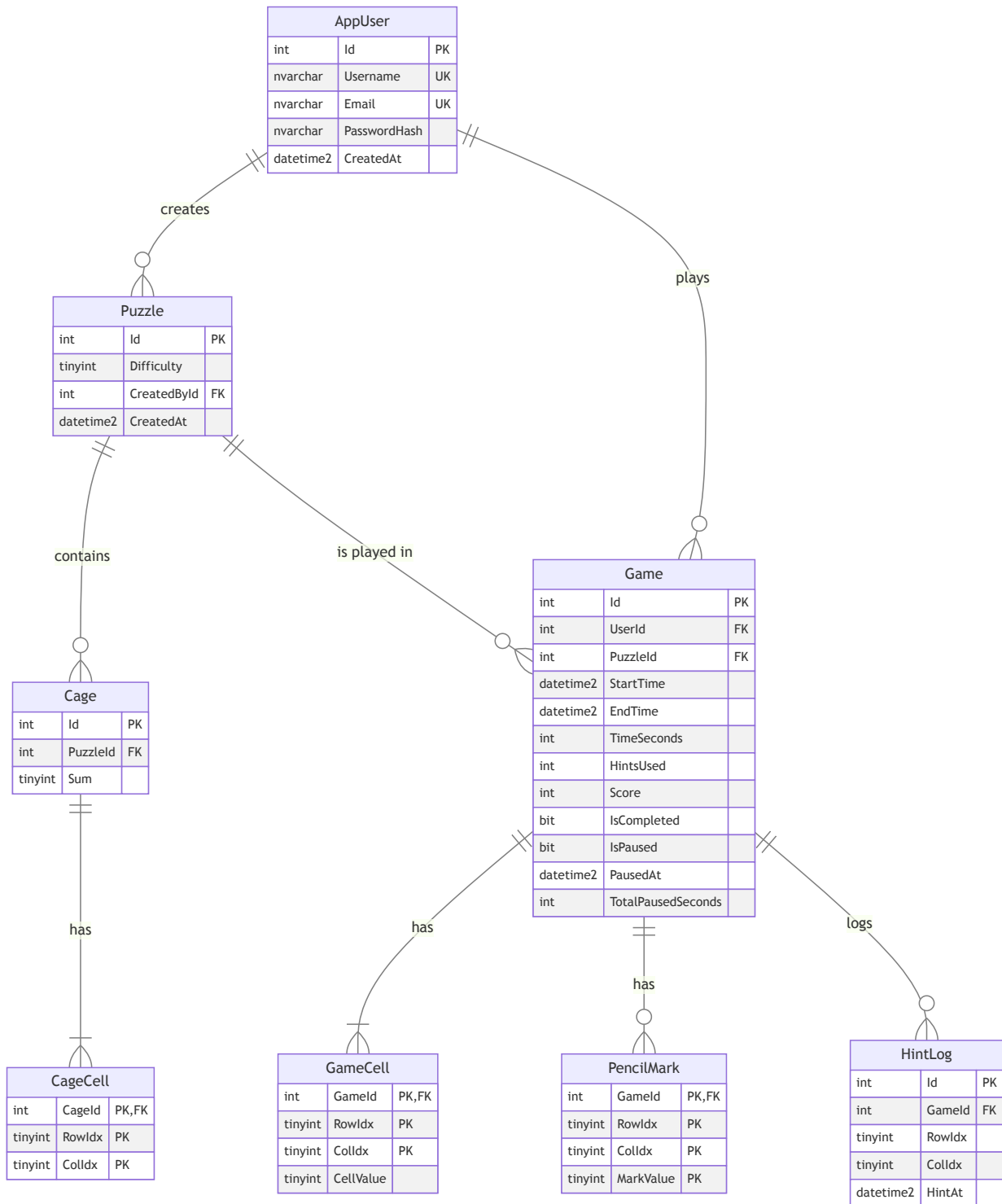
| Aspekt                          | Details                                                                                                                                                                                                                                                                                                                |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Zweck</b>                    | Reine Algorithmik für die vier Killer-Sudoku-Regeln aus README §1: (1) "Each row, column, and nonet contains each number exactly once", (2) "The sum of all numbers in a cage must match the small number printed in its corner", (3) "No number appears more than once in a cage", (4) "The solution must be unique". |
| <b>Eingehende Schnittstelle</b> | <code>ISolverService</code> — Methoden <code>Solve(givenValues, cages)</code> und <code>CountSolutions(givenValues, cages)</code> ( <b>Funktionalitäts-Matrix</b> §Service-Interfaces).                                                                                                                                |
| <b>Ausgehende Schnittstelle</b> | <b>Keine</b> — pure C# ohne Framework- oder DB-Abhängigkeit.                                                                                                                                                                                                                                                           |
| <b>Enthaltene Klassen</b>       | <code>SolverService</code> (Backtracker + Constraint-Propagation) · <code>SolutionValidator</code> (Row/Col/Nonet/Cage-Check für UC09 §6.3) · <code>HintStrategies</code> (Naked Single, Cage-Forced; siehe UC07 in <b>Use-Cases-Dokument</b> ) · interne Records <code>CageDef</code> , <code>Board</code> .          |
| <b>Performance-Anforderung</b>  | AC11.2: terminiert auch bei schweren Puzzles in < 2 s; bricht bei Zähl-Modus nach der 2. gefundenen Lösung ab.                                                                                                                                                                                                         |
| <b>Test-Strategie</b>           | Reine Unit-Tests mit den 2 README-Beispielen (§1.2) + Edge-Cases: unsolvable, multi-solution, vollständig vor-gelöst, minimal-clue. Höchste Test-Coverage-Priorität (siehe <b>Funktionalitäts-Matrix</b> §Kritische Abhängigkeiten).                                                                                   |

## 5.2.4 KillerSudoku.Data — Persistence Layer

| Aspekt                                | Details                                                                                                                                                                                                                                                |
|---------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Zweck</b>                          | EF-Core-Persistenz; Mapping C#-Entities ↔ T-SQL-Tabellen aus <b>Datenbank-Skript</b> ; Query-Optimierung; View-Zugriff.                                                                                                                                |
| <b>Eingehende Schnittstelle</b>       | <code>SudokuDbContext</code> ( <code>DbSet&lt;T&gt;</code> für jede Entity).                                                                                                                                                                           |
| <b>Ausgehende Schnittstelle</b>       | T-SQL gegen MS-SQL Express via ADO.NET-Provider.                                                                                                                                                                                                       |
| <b>Enthaltene Entities</b>            | <code>AppUser</code> · <code>Puzzle</code> · <code>Cage</code> · <code>CageCell</code> · <code>Game</code> · <code>GameCell</code> · <code>PencilMark</code> · <code>HintLog</code> . Schema-Quelle: <b>Datenbank-Skript</b> , ERD: <b>ER-Modell</b> . |
| <b>View-Read-Model</b>                | <code>vw_Highscore</code> (siehe <b>Datenbank-Skript</b> Zeilen 215–231) — wird per <code>Set&lt;HighscoreEntry&gt;.FromSqlRaw(...)</code> oder Keyless-Entity gelesen.                                                                                |
| <b>DB-Constraints (Defense-Layer)</b> | Siehe §5.4 unten.                                                                                                                                                                                                                                      |
| <b>Test-Strategie</b>                 | Integration-Tests mit Testcontainers (MS-SQL-Image) oder lokaler Test-DB; Constraint-Tests (INSERT mit Difficulty=4 muss fehlschlagen) gemäß <b>ER-Modell</b> §Ableitungen für Tests.                                                                  |

## 5.3 ER-Modell (Verweis + Einbettung)

Das ER-Modell ist autoritativ in **ER-Modell** dokumentiert, inklusive Design-Entscheidungen, Constraints und Sample-Queries. Für die PDF-Lesbarkeit ist das Mermaid-ERD hier eingebettet:



### Schlüssel-Begründungen (Auszug aus ER-Modell):

- `AppUser` statt `User` — `USER` ist reserviertes T-SQL-Keyword.
- `RowIdx` / `ColIdx` statt `Row` / `Col` — beides reserviert.
- **Keine Solution-Spalte** in `Puzzle` — UC04 wörtlich: *"No solution is recorded. Solutions must be calculated with an algorithm"*.
- `Highscore` als **VIEW** statt denormalisierte Tabelle — single source of truth, kein Sync-Problem.

- Pause-Mechanik via `IsPaused` + `PausedAt` + `TotalPausedSeconds` → erlaubt sauberen Resume mit `TimeSeconds = DATEDIFF(SECOND, StartTime, EndTime) - TotalPausedSeconds`.

## 5.4 SQL-Schema-Übersicht

Vollständiges Schema in **Datenbank-Skript**. Folgende Objekte werden angelegt:

### Tabellen (in Reverse-FK-Reihenfolge erzeugt):

- `AppUser` — Benutzer mit gehashtem Passwort
- UNIQUE: `Username`, `Email`
- CHECK: `Username` non-empty, `Email` LIKE `%_@_%.__%`
- `Puzzle` — Killer-Sudoku-Definition, **keine** Solution-Spalte
- FK: `CreatedById` → `AppUser(Id)`
- CHECK: `Difficulty` BETWEEN 1 AND 3
- `Cage` — Cage-Gruppe mit Soll-Summe
- FK: `PuzzleId` → `Puzzle(Id)` ON DELETE CASCADE
- CHECK: `Sum` BETWEEN 1 AND 45
- Index: `IX_Cage_PuzzleId`
- `CageCell` — Zelle in einem Cage (Composite PK)
- PK: `(CageId, RowIdx, ColIdx)`
- FK: `CageId` → `Cage(Id)` ON DELETE CASCADE
- CHECK: `RowIdx`, `ColIdx` BETWEEN 0 AND 8
- `Game` — Spiel-Session
- FK: `UserId` → `AppUser(Id)`, `PuzzleId` → `Puzzle(Id)`
- CHECK: `TimeSeconds` ≥ 0, `HintsUsed` ≥ 0, `Score` ≥ 0, `TotalPausedSeconds` ≥ 0
- Indices: `IX_Game_UserId`, `IX_Game_PuzzleId`
- `GameCell` — Aktueller Spielzustand pro Zelle
- PK: `(GameId, RowIdx, ColIdx)`
- FK: `GameId` → `Game(Id)` ON DELETE CASCADE
- CHECK: `CellValue` IS NULL OR BETWEEN 1 AND 9
- `PencilMark` — Kandidaten-Annotationen (UC14)
- PK: `(GameId, RowIdx, ColIdx, MarkValue)`
- FK: `GameId` → `Game(Id)` ON DELETE CASCADE
- CHECK: `MarkValue` BETWEEN 1 AND 9
- `HintLog` — Audit-Log für Hint-Nutzung (UC07)
- FK: `GameId` → `Game(Id)` ON DELETE CASCADE
- Index: `IX_HintLog_GameId`

### Index/Constraint-Spezialitäten:

- **Trigger** `trg_CageCell_UniquePerPuzzle` — verhindert, dass eine Zelle in mehreren Cages desselben Puzzles liegt. Wirft `THROW 50001` bei Verletzung (siehe **Datenbank-Skript** Zeilen 100–121).
- **Filtered Unique Index** `UX_Game_ActiveOnly` — `(UserId, PuzzleId) WHERE IsCompleted = 0`; setzt AC13.3 durch ("max 1 aktives Game pro User-Puzzle-Kombi").

#### View:

- `vw_Highscore` — JOIN über `Game ⋈ AppUser ⋈ Puzzle` für `IsCompleted = 1 AND Score IS NOT NULL`. Liefert `GameId · UserId · Username · PuzzleId · Difficulty · TimeSeconds · HintsUsed · Score · CompletedAt`. Wird von `IHighscoreService.GetTopAsync(limit)` für UC08 verwendet.

---

**Nächstes Kapitel:** [Kapitel 6 — Laufzeitsicht](#) — Sequenz-Diagramme der vier wichtigsten Szenarien (Puzzle-Save, Hint, Check-Solution, Pause/Resume).

## 6 Laufzeitsicht

---

arc42 v8.2 · Kapitel 6 · Killer Sudoku Bezug: [Kapitel 5](#) · **Use-Cases-Dokument** · **Funktionalitäts-Matrix** · **Validation-Regeln** · **ER-Modell**

Dieses Kapitel beschreibt die wichtigsten **Laufzeit-Szenarien** als Sequenz-Diagramme. Service-Methoden-Namen entsprechen exakt dem Service-Interface-Block in **Funktionalitäts-Matrix** §Service-Interfaces. DB-Tabellen sind in **Datenbank-Skript** belegt.

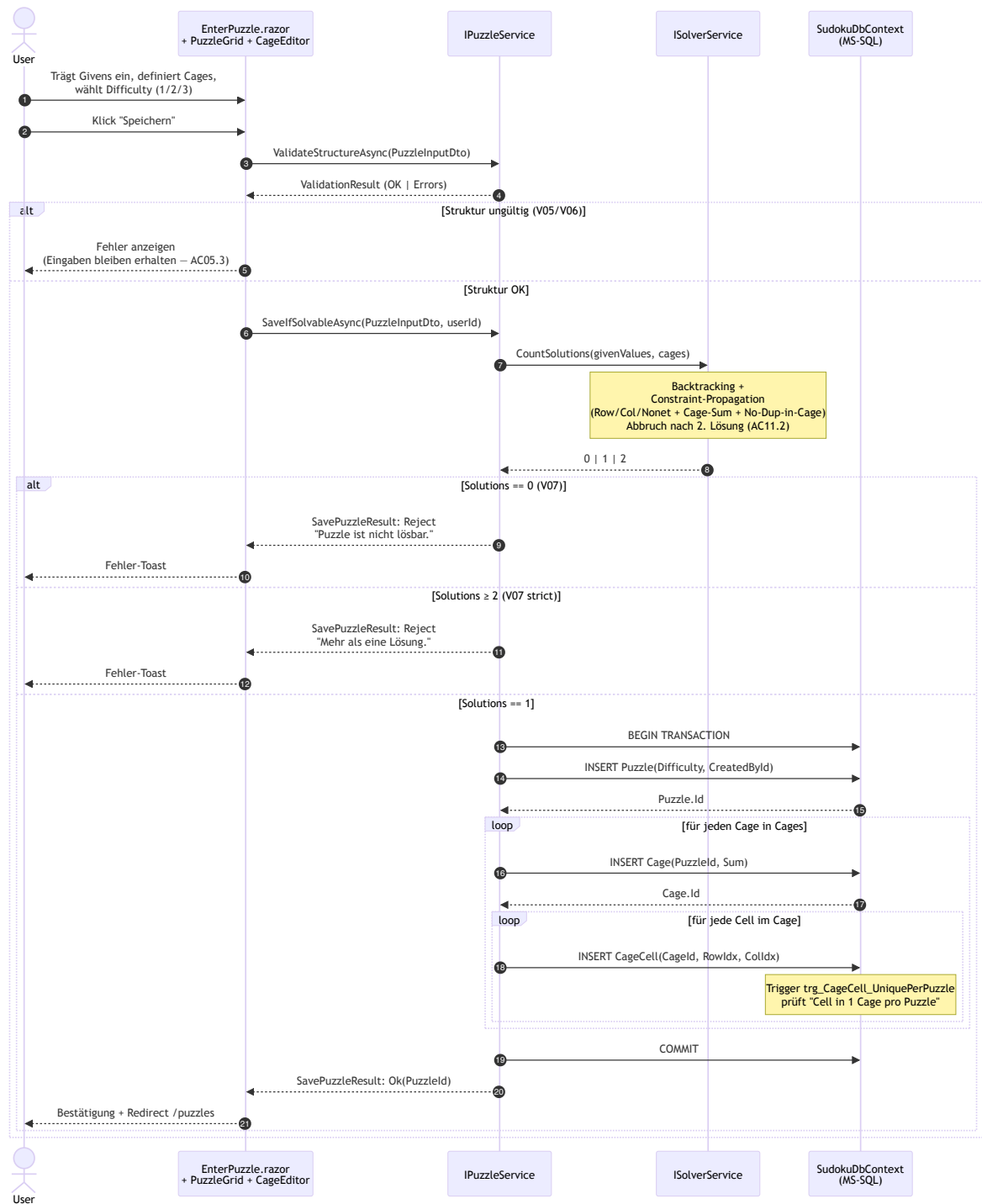
**Auswahl der Szenarien:** Die vier dargestellten Use-Case-Flows decken die kritischen Architektur-Pfade ab — Save-with-Solver, Hint-Generation, Solution-Check (inkl. Sum-Fast-Fail) und Pause/Resume-State-Persistierung. Diese Pfade involvieren alle drei Layer + den Domain-Kern.

---

### 6.1 Szenario A — Puzzle anlegen und speichern (UC04 + UC05)

**Ziel:** User erstellt ein neues Killer-Sudoku, Server validiert Struktur, ruft Solver für Eindeutigkeits-Check, persistiert nur bei genau einer Lösung. Wörtliche README-Grundlage: *"The puzzle can only be saved if it is solvable"* (UC05) und *"The solution must be unique"* (§1).

**Beteiligt:** UC04 + UC05 · **V05** · **V06** · **V07** · Screen S5.



### Wichtige Architektur-Eigenschaften:

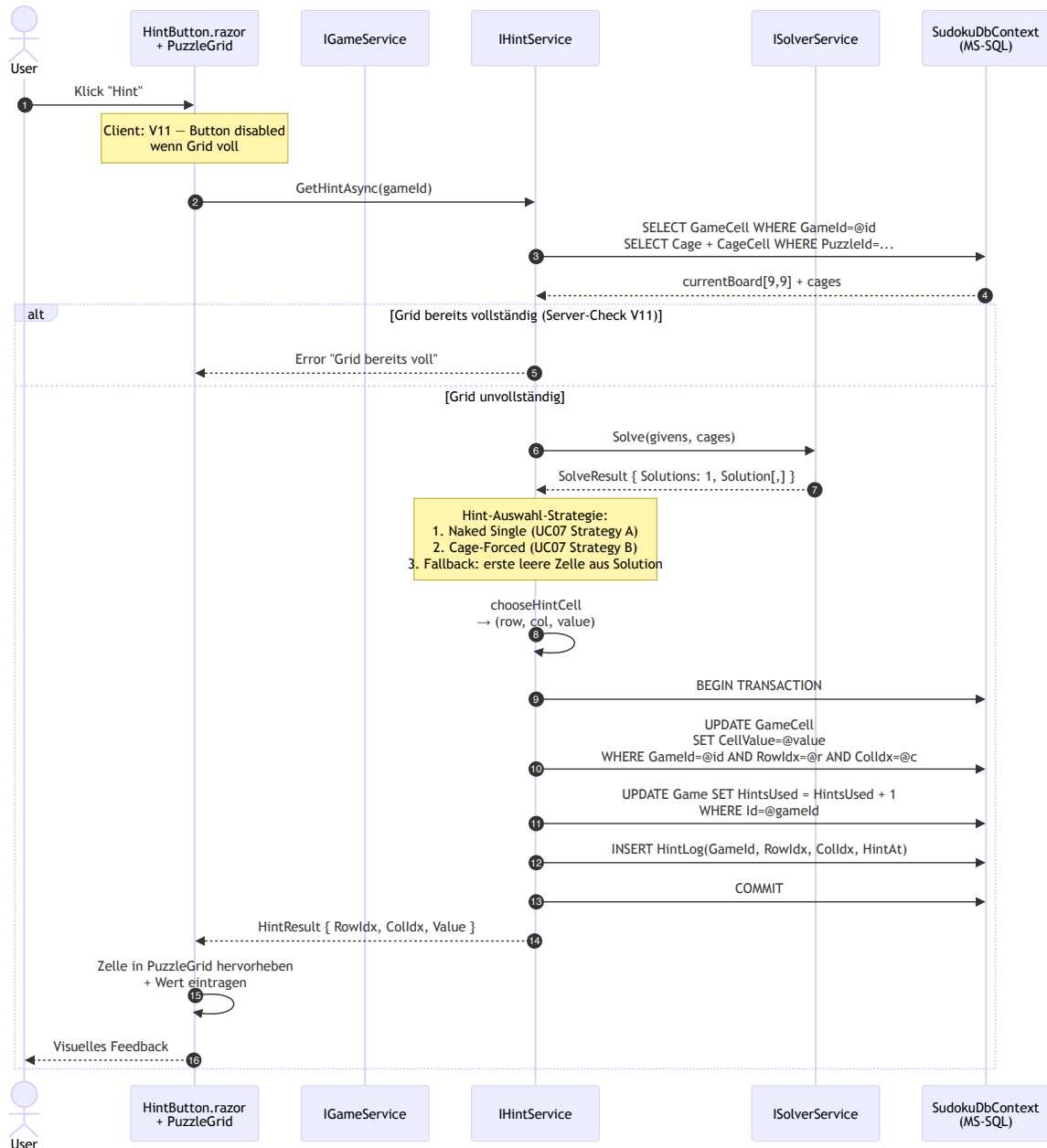
- **Transaktionsklammer** liegt in `IPuzzleService.SaveIfSolvableAsync`, nicht in der UI.
- **Solver wird VOR jedem INSERT** ausgeführt — keine "Roll-Back-on-Failure"-Konstruktion nötig.
- **Keine Solution-Speicherung** — UC04 wörtlich: *"No solution is recorded"*; die Cage-Tabelle reicht für späteres Re-Solving.
- **DB-Trigger** ist Last-Line-of-Defense — falls die Application-Validierung Bugs hat, schützt der Trigger Datenintegrität (Defense-in-Depth).



## 6.2 Szenario B — Hint anfordern (UC07)

**Ziel:** User klickt "Hint", System bestimmt aus aktuellem Spielzustand eine Zelle + den korrekten Wert, persistiert Hint-Audit-Eintrag, inkrementiert `HintsUsed` (Score-relevant via UC08).

**Beteiligt:** UC07 · V11 · Screen S6 · `IHintService` + `ISolverService`.



### Wichtige Architektur-Eigenschaften:

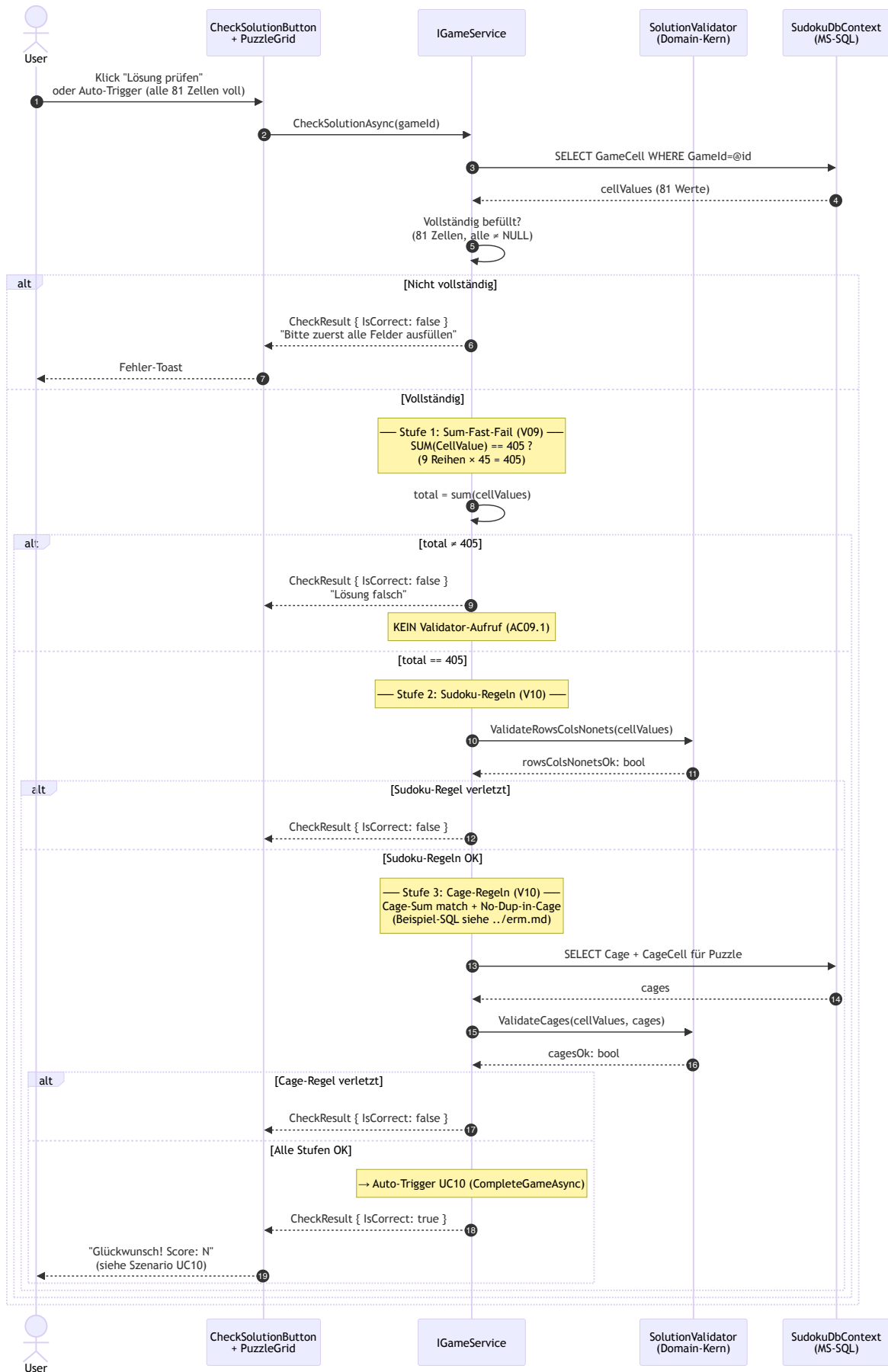
- **Solver wird stateless aufgerufen** — er bekommt den aktuellen Board-Zustand inkl. User-Eingaben; durch AC11.1 (Solver erkennt  $0/1/\geq 2$  korrekt) ist garantiert, dass das gespeicherte Puzzle eine eindeutige Lösung hat, also `Solutions == 1` für vorherige Hint-Anfragen sicher ist (solange User-Eingaben nicht widersprüchlich sind — siehe UC07 AC07.3).
- **HintsUsed** wird atomar mit dem `GameCell`-Update und dem `HintLog`-INSERT in einer Transaktion erhöht — Audit + Score-Input bleiben konsistent.

- **Score-Wirkung:** Jeder Hint kostet 300 Punkte (Formel UC08 §High Score) — daher die Audit-Disziplin.
- 

## 6.3 Szenario C — Lösung prüfen (UC09)

**Ziel:** User triggert Lösungsprüfung; System validiert in **drei Stufen**, sortiert nach Performance: (1) Sum-Check 405 als O(1)-Fast-Fail, (2) Row/Col/Nonet-Check, (3) Cage-Sum + Cage-No-Duplicate. README §2.3 wörtlich: *"Use the value for a 'simple' validation before checking the solution with an algorithm"*.

**Beteiligt:** UC09 · **V09** · **V10** · Screen S6 · `IGameService` .



Performance-Begründung (in Reihenfolge der drei Stufen):

1. **Sum-Check 405** —  $O(81)$  Addition, kein Allokation. Filtert grobe Fehler (falsche Zahl, vergessene Zelle) sofort.
2. **Row/Col/Nonet** —  $27 \times 9$ -Element-Distinct-Check,  $O(243)$  ohne Set-Allokation via Bitmask.
3. **Cage-Sum + No-Duplicate** — Pro Cage einmal `cellValues` durchgehen, Set-basierter Duplikat-Check. Komplexität proportional zur Anzahl Cages  $\times$  Cage-Größe  $\leq 81$  Operationen.

Die Sum-Check-Reihenfolge ist nicht Optimierung um der Optimierung willen — sie ist **wörtliche Anforderung** aus README §2.3 und Pattern für **alle** "simple before algorithmic" Validierungen im System.

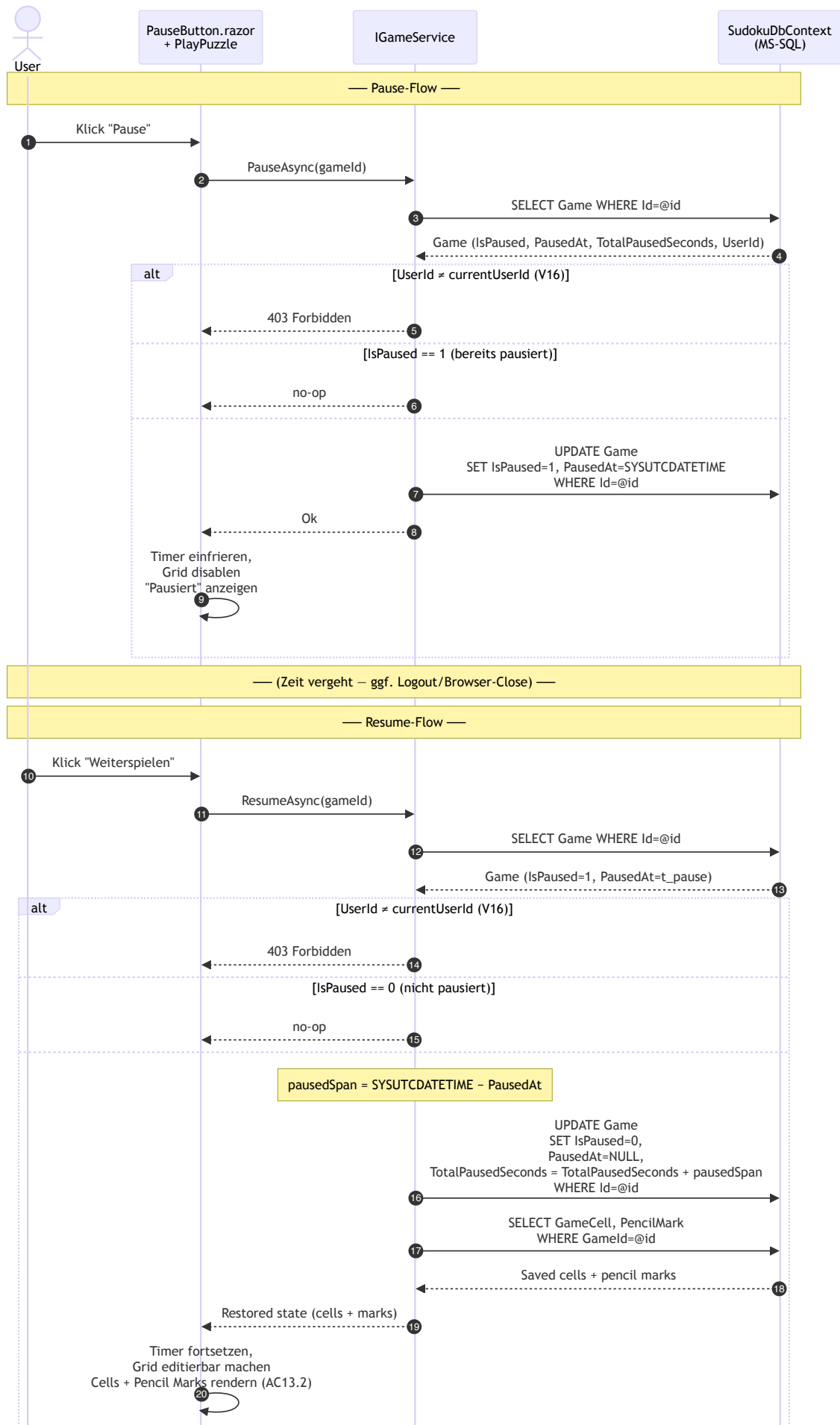
---

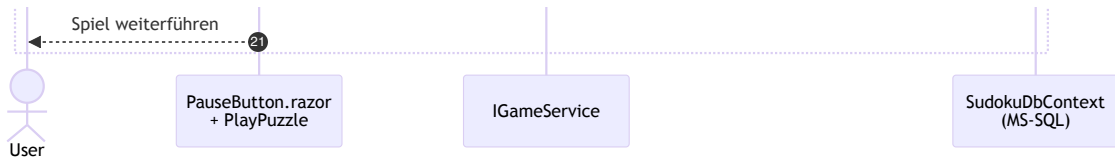
## 6.4 Szenario D — Pause und Resume (UC13)

**Ziel:** User pausiert ein laufendes Spiel; Server friert Timer ein und persistiert den Zustand; bei Resume wird der Timer um die Pausen-Dauer korrigiert weitergeführt. Formel aus **ER-Modell**:

`TimeSeconds = DATEDIFF(SECOND, StartTime, EndTime) - TotalPausedSeconds` .

**Beteiligt:** UC13 · **V12** · Screen S6 · `IGameService.PauseAsync` / `ResumeAsync` .





### Wichtige Architektur-Eigenschaften:

- **TotalPausedSeconds** wird inkrementell aufaddiert — mehrere Pause/Resume-Zyklen sind dadurch sauber unterstützt.
- **Timer-Berechnung** bleibt in der DB-Formel verankert (siehe Comment-Block in **Datenbank-Skript** Zeilen 125–127): bei Spiel-Abschluss (UC10) gilt `TimeSeconds = DATEDIFF(SECOND, StartTime, EndTime) - TotalPausedSeconds`.
- **Filtered Unique Index** `UX_Game_ActiveOnly` (**ER-Modell**) garantiert AC13.3: ein User kann pro Puzzle nur **ein** nicht abgeschlossenes Game haben — verhindert Pause-im-Spiel-1 + Neu-Start-Spiel-2-vor-Resume-Doubles.
- **Auto-Save bei jedem Move** (UC13 Alt-Flow "Browser-Tab schliessen ohne Pause"): `IGameService.SetCellValueAsync` persistiert jeden Zell-Change sofort — daher ist auch ein impliziter "Pause durch Wegnavigation" verlustfrei.
- **Authorization (V16)**: jede Mutation prüft `Game.UserId == currentUserId` und antwortet mit 403 bei Mismatch.

**Nächstes Kapitel:** [Kapitel 7 — Verteilungssicht](#) (TBD) — Deployment-Diagramm: lokale Dev-Workstation, MS-SQL-Express-Instanz, Build-/Run-Konfiguration.

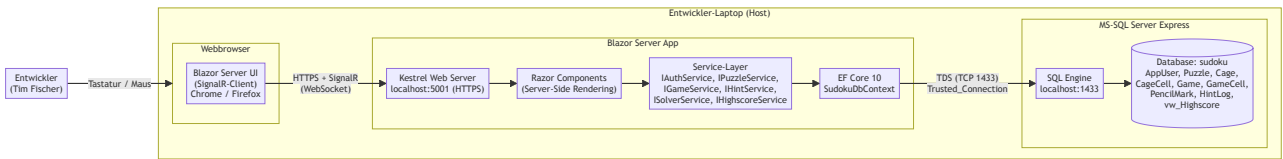
# 7 Verteilungssicht

arc42 v8.2 · Skills Battle 2026 — Application Development — Killer Sudoku Quelldokument  
(autoritativ): **Aufgabenstellung**

Dieses Kapitel beschreibt die physische Verteilung der Anwendung. Da die Aufgabenstellung explizit ein lokales Setup vorsieht (README §1.3: "create a database named 'sudoku' on MySQL or MS-SQL server **running on your machine**"), ist die Verteilungssicht bewusst einfach gehalten: Alle Komponenten laufen auf demselben Entwickler-Laptop.

## §7.1 Deployment-Diagramm

Die folgende Verteilungssicht zeigt die Laufzeit-Knoten der Anwendung beim Wettbewerbs-Setup. Alle drei Komponenten (Browser, Blazor-Server-App, DB-Server) laufen auf demselben Host (Entwickler-Laptop).



## Knoten-Inventar

| Knoten                | Typ               | Adresse / Port         | Technologie                               | Quelle                                                 |
|-----------------------|-------------------|------------------------|-------------------------------------------|--------------------------------------------------------|
| Entwickler-Laptop     | Hardware-Host     | localhost              | macOS / Windows                           | README §1.3 ("on your machine")                        |
| Webbrowser            | Client-Runtime    | —                      | Chrome oder Firefox (modern HTML5)        | Mockup-Briefs (Web-UI)                                 |
| Blazor Server App     | Anwendungs-Server | localhost:5001 (HTTPS) | .NET 10, Kestrel, Blazor Server (SignalR) | <b>ER-Modell, Funktionalitäts-Matrix</b>               |
| MS-SQL Server Express | DB-Server         | localhost:1433         | MS-SQL Express, Datenbank <b>sudoku</b>   | README §1.3 ("MS-SQL server"), <b>Datenbank-Skript</b> |

## Kommunikationspfade

| Von                  | Zu                   | Protokoll                         | Bemerkung                                                                                                            |
|----------------------|----------------------|-----------------------------------|----------------------------------------------------------------------------------------------------------------------|
| Browser              | Blazor<br>Server App | HTTPS +<br>WebSocket<br>(SignalR) | Blazor Server hält permanente SignalR-Verbindung pro Tab/Circuit (siehe <b>Funktionalitäts-Matrix</b> )              |
| Blazor<br>Server App | MS-SQL<br>Express    | TDS (TCP 1433)                    | EF Core via <code>SqlConnection</code> , <code>Trusted_Connection</code> (Windows-Auth) bzw. lokaler Service-Account |

## Bemerkungen

- Es gibt **keine** Cloud-Komponenten, kein CDN, keine externen APIs — die Aufgabenstellung sieht ein rein lokales Setup vor.
- Die SignalR-Verbindung ist Folge des Blazor-Server-Modells (siehe [ADR-001](#)).
- Single-Host-Setup bedeutet: kein Reverse-Proxy, keine Lastverteilung, keine separate Auth-Komponente.

## §7.2 Infrastruktur-Anforderungen (lokal)

### Software-Voraussetzungen

| Komponente                                 | Version                      | Zweck                                                                     | Quelle                                                            |
|--------------------------------------------|------------------------------|---------------------------------------------------------------------------|-------------------------------------------------------------------|
| .NET SDK                                   | 10.x                         | Build & Run der Blazor-Server-App                                         | Stack-Definition <b>ER-Modell</b> , <b>Funktionalitäts-Matrix</b> |
| MS-SQL Server Express                      | aktuelle Version             | Datenbank-Engine für <code>sudoku</code> -DB                              | README §1.3 ("MS-SQL server")                                     |
| SQL Server Management Studio (SSMS)        | beliebig                     | Ausführen von <b>Datenbank-Skript</b> (Schema-Setup), Tabellen-Inspektion | Operative Empfehlung                                              |
| Visual Studio 2026 oder Visual Studio Code | beliebig                     | Entwicklungs-IDE                                                          | Empfehlung Entwickler-Werkzeug                                    |
| Webbrowser                                 | Chrome oder Firefox (modern) | Frontend-Runtime                                                          | Standard Blazor-Server-Anforderung                                |



## Hardware-Voraussetzungen

Da Blazor Server serverseitig rendert und der Solver eine reine In-Memory-Operation ist, sind die Hardware-Anforderungen moderat:

- Standard-Entwickler-Laptop (8 GB RAM, x64-CPU) ausreichend
- Disk: < 1 GB für SDK + Express + Source-Code + DB-Files

## Konfiguration / Ports

| Port | Dienst                        | Bemerkung                       |
|------|-------------------------------|---------------------------------|
| 5001 | Kestrel HTTPS (Blazor Server) | Default-Port .NET-Web-Templates |
| 1433 | MS-SQL Server Express         | Default-Port SQL-Server         |

## Datenbank-Setup

Die Datenbank-Erstellung erfolgt einmalig via SSMS oder `sqlcmd` :

```
# Beispiel mit sqlcmd (Windows / macOS-Crossplattform)
sqlcmd -S "(local)\SQLEXPRESS" -E -i db/sudoku.sql
```

Das Skript **Datenbank-Skript** ist **idempotent**: Es prüft auf `DB_ID('sudoku')` und droppt bestehende Objekte in Reverse-FK-Reihenfolge, bevor neu angelegt wird. Damit ist ein wiederholtes Ausführen ohne Konflikte möglich.

## §7.3 Build & Run

### Build

```
# Restore + Build
dotnet restore
dotnet build -c Release
```

### Run (Entwicklung)

```
dotnet run --project src/Sudoku.Web
```

Die App ist anschliessend unter `https://localhost:5001` erreichbar.

## Connection-String

```
Server=(local)\SQLEXPRESS;Database=sudoku;Trusted_Connection=True;TrustServerCertificate=
```

Der Connection-String wird via `appsettings.json` / `appsettings.Development.json` oder Umgebungsvariable konfiguriert. Trusted Connection (Windows-Auth) ist auf dem Wettbewerbs-Laptop ausreichend; Cross-Plattform-Setups (macOS) verwenden Username/Password über `User Id=... ;Password=...`.

**Sicherheit:** Auch im lokalen Setup wird der Connection-String nicht in den Source-Code hardkodiert (siehe [Kapitel 8 §8.4 Logging](#) und **V03**). `TrustServerCertificate=True` ist akzeptabel, da kein Netzwerk-Hop über das lokale Setup hinaus stattfindet.

## Tests ausführen

```
dotnet test
```

Test-Frameworks gemäss [ADR-011](#):

- Unit-Tests (xUnit) — Solver-Logik, Service-Methoden, Validation
- Component-Tests (bUnit) — Razor-Components (**Funktionalitäts-Matrix**)
- Integration-Tests (xUnit + `WebApplicationFactory`) — DB + Auth + Service-End-to-End
- E2E-Tests (Playwright .NET) — Kritische User-Flows (UC01, UC03, UC06, UC09)

## §7.4 Submission-Artefakte

### ZIP-Inhalt (gemäss README §1.4)

README §1.4 fordert wörtlich:

"All deliverables must be submitted in a zip file named `AppDev_Name_FirstName.zip`. The deliverables are the documentation including the test protocol, executable files, the source code and the database script."

Daraus leitet sich folgende Submission-Struktur ab:

```

AppDev_Fischer_Tim.zip
├── doc/
│   ├── arc42.pdf (Kapitel 01–12 als ein PDF)
│   ├── mockups/ (PNG-Exports aus Figma)
│   └── test-protocol.pdf
├── src/ (vollständiger .NET-Source)
│   ├── Sudoku.Web/ (Blazor Server App)
│   ├── Sudoku.Domain/ (Solver, Domain-Modelle)
│   ├── Sudoku.Data/ (EF Core, Repositories)
│   ├── Sudoku.Tests.Unit/
│   ├── Sudoku.Tests.Integration/
│   └── Sudoku.Tests.E2E/
├── bin/
│   └── Release/net10.0/ (Build-Output, ausführbare Files)
├── db/
│   └── sudoku.sql (DB-Skript)

```

## Build-Output (Executables)

Das README fordert "executable files". Für .NET 10 + Blazor Server bedeutet dies:

```
dotnet publish -c Release -o bin/Release/net10.0
```

Der Publish-Output enthält:

- `Sudoku.Web.dll` (Haupt-Assembly)
- `Sudoku.Web.exe` (Windows-Launcher, falls auf Windows publiziert)
- Alle Dependency-DLLs
- `appsettings.json` (mit Connection-String-Slot)
- `wwwroot/` (statische Assets, CSS, JS)

Starten der publizierten App:

```
cd bin/Release/net10.0
dotnet Sudoku.Web.dll
```

## Datenbank-Skript

`db/sudoku.sql` (siehe **Datenbank-Skript**) ist im ZIP enthalten und entspricht README §1.3:

"Create for each table a SQL statement and save them in a script called **Datenbank-Skript**. Don't forget to create the primary and foreign key constraints."

Das Skript erstellt:

- Datenbank `sudoku` (idempotent)
- Tabellen `AppUser`, `Puzzle`, `Cage`, `CageCell`, `Game`, `GameCell`, `PencilMark`, `HintLog`
- PK/FK/CHECK/UNIQUE-Constraints
- Trigger `trg_CageCell_UniquePerPuzzle` (siehe [ADR-004](#))
- View `vw_Highscore` (siehe [ADR-003](#))

## Dokumentation

README §2.6 fordert wörtlich folgende Dokumentations-Sektionen:

| README-Anforderung              | Abgedeckt in                                                                   |
|---------------------------------|--------------------------------------------------------------------------------|
| "Mockup"                        | <b>Mockup-Briefings + Mockup-Verzeichnis</b>                                   |
| "Database diagram"              | <b>ER-Modell</b> (Mermaid ERD) + Kapitel 5 Bausteinsicht                       |
| "Class diagram"                 | Kapitel 5 Bausteinsicht — Service-Interfaces aus <b>Funktionalitäts-Matrix</b> |
| "Additionally chosen use cases" | <b>Use-Cases-Dokument</b> UC12–UC14 + <a href="#">ADR-014</a>                  |
| "Validation rules"              | <b>Validation-Regeln</b> (V01–V16)                                             |
| "Test protocol"                 | <b>Test-Protokoll + Test-Protokoll</b>                                         |

README §2.6 weiter: "For the delivery, create a PDF document." Die arc42-Kapitel werden zu einem PDF zusammengeführt (Submission-Format).

## Termin-Constraints

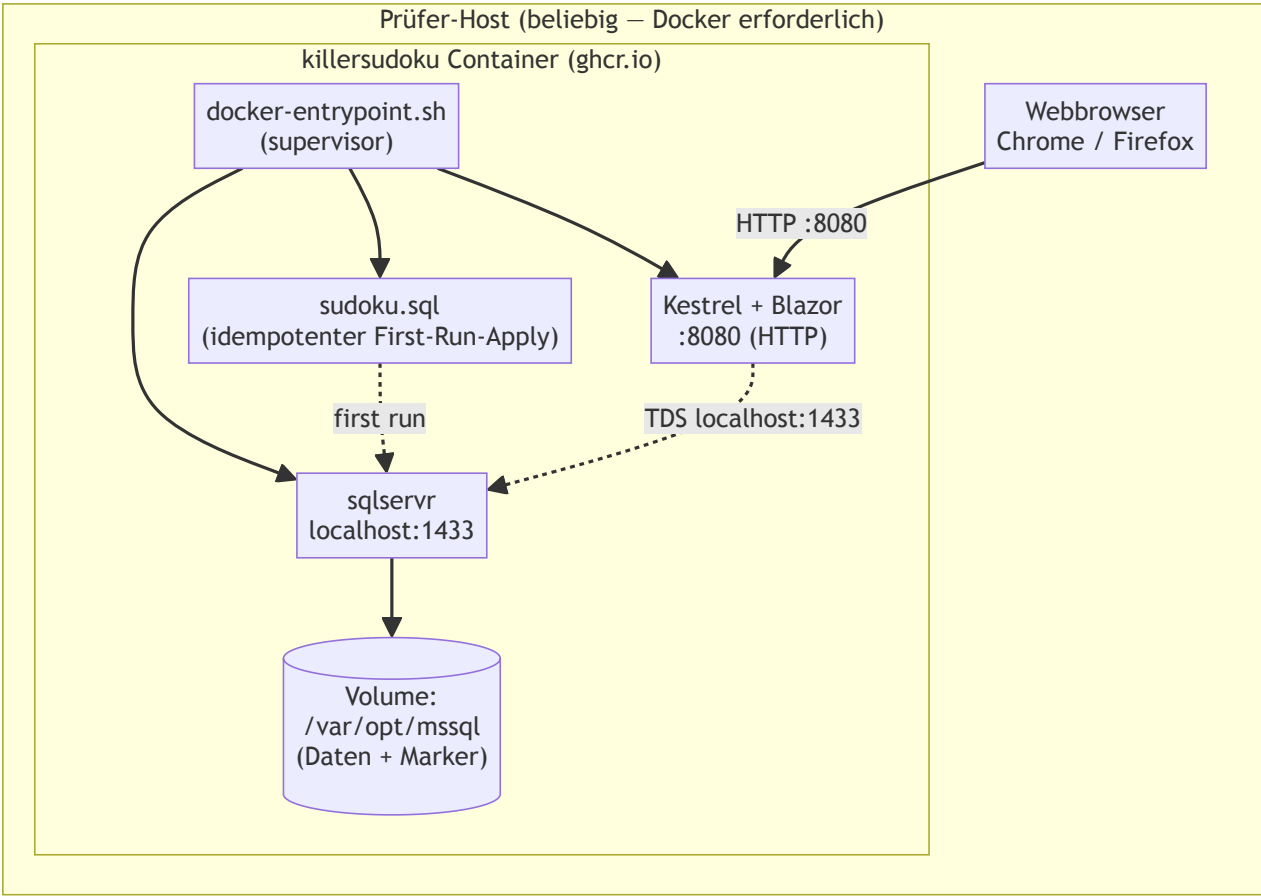
README §1.4: "Note that the submission of the planned test cases must take place **by 11:30 o'clock**." Der Test-Plan ist in **Test-Protokoll** dokumentiert und wird vor 11:30:00 abgegeben (siehe Qualitätsziel Q3 in [Kapitel 1 §1.2](#)).

## §7.5 Container-Deployment (Single-Image Docker)

Zusätzlich zur nativen Variante (§7.1–§7.3) wird ein **containerisiertes Single-Image-Deployment** ausgeliefert. Es enthält Webserver, App und Datenbank in einem Image — der Prüfer kann mit einem einzigen `docker run` die gesamte Anwendung starten, ohne MS-SQL Server lokal installieren zu müssen.

Siehe [ADR-015](#) für die Begründung der Single-Image-Wahl (statt klassischem Multi-Service-Compose).

## Deployment-Diagramm (Container)



## Image-Inhalt

| Layer                                | Quelle                                                                 | Zweck                                                 |
|--------------------------------------|------------------------------------------------------------------------|-------------------------------------------------------|
| Base                                 | <code>mcr.microsoft.com/mssql/server:2022-latest</code> (Ubuntu 22.04) | DB-Engine + EULA-Akzeptanz                            |
| <code>aspnetcore-runtime-10.0</code> | <code>packages.microsoft.com</code>                                    | .NET 10 ASP.NET Runtime für Blazor Server             |
| <code>mssql-tools18</code>           | MS-Repo                                                                | <code>sqlcmd</code> für Schema-Appl beim ersten Start |
| <code>/app/</code>                   | Multi-Stage Build (Stage 1)                                            | <code>dotnet publish</code> Output des Web-Projekts   |
| <code>/docker-init/sudoku.sql</code> | Repo <code>db/sudoku.sql</code>                                        | Schema-Script (DDL + Trigger + View)                  |
| <code>/docker-entrypoint.sh</code>   | Repo-Root                                                              | Startup-Orchestrierung                                |

## Startup-Sequenz

`docker-entrypoint.sh` führt beim Container-Start folgende Schritte aus (atomar, vollständige Konvergenz):

1. `sqlservr` als Background-Prozess starten
2. Bis zu 60x `sqlcmd -Q "SELECT 1" -Ping (≤ 2 Min)` — wartet auf DB-Readiness
3. Falls Marker-File `/var/opt/mssql/.killersudoku-schema-applied` **nicht** existiert: **Datenbank-Skript** ausführen und Marker setzen
4. `dotnet KillerSudoku.Web.dll` als zweiter Hintergrund-Prozess starten
5. `wait -n` blockiert bis SQL oder App terminiert → dann sauberer SIGTERM an beide Children

## Build & Run

### Option A — Pull vom Registry (schnellster Pfad für Prüfer)

```
docker pull ghcr.io/tim-fischer-zh/killer-sudoku:latest
docker run -d \
  --name killersudoku \
  -p 8080:8080 \
  -e MSSQL_SA_PASSWORD='Sudoku!Strong#Pass#2026' \
  -v killersudoku-data:/var/opt/mssql \
  ghcr.io/tim-fischer-zh/killer-sudoku:latest

# Browser → http://localhost:8080
```

### Option B — Lokal aus Sources bauen

```
# Repo-Root als Build-Context (enthält source/ + db/ + docker-entrypoint.sh)
docker build -t killersudoku:local .
docker run -d -p 8080:8080 \
  -e MSSQL_SA_PASSWORD='Sudoku!Strong#Pass#2026' \
  killersudoku:local
```

### Option C — `docker compose` (empfohlen)

```
docker compose up --build
```

Compose-Datei: **docker-compose.yml** — bindet Port 8080 (App) und 1433 (DB, optional für externe SSMS-Inspektion), persistiert SQL-Daten in einem Named Volume.

## Ports

| Port | Zweck                                      | Default-Mapping                        |
|------|--------------------------------------------|----------------------------------------|
| 8080 | Kestrel HTTP (Blazor Server)               | -p 8080:8080                           |
| 1433 | MS-SQL (intern + optional extern für SSMS) | -p 1433:1433 (kann weggelassen werden) |

**Hinweis:** Im Container läuft Kestrel auf HTTP ( :8080 ), nicht HTTPS. Ein TLS-Terminator (z.B. Traefik/Caddy) ist nicht im Image enthalten — für die Submission-Bewertung ist Localhost-HTTP ausreichend. Die Auth-Cookies bleiben `HttpOnly + SameSite=Lax` (siehe **V03**) — der `Secure`-Flag ist im Container-Lauf nur unter HTTPS-Reverse-Proxy zu setzen.

## Environment-Variablen

| Variable                  | Default                 | Zweck                                                             |
|---------------------------|-------------------------|-------------------------------------------------------------------|
| MSSQL_SA_PASSWORD         | Sudoku!Strong#Pass#2026 | SA-Passwort (Strong-Password-Policy von MS-SQL muss erfüllt sein) |
| MSSQL_PID                 | Express                 | Edition (Free-Tier für Submission)                                |
| ACCEPT_EULA               | Y                       | MS-SQL Lizenz-Akzeptanz (Pflicht)                                 |
| ASPNETCORE_URLS           | http://+:8080           | Kestrel-Binding                                                   |
| ConnectionStrings__Sudoku | siehe Dockerfile        | Override möglich für Tests                                        |

## Persistenz

Die DB-Files ( `mdf` / `ldf` ) liegen in `/var/opt/mssql` im Container. Ein Named-Volume `killersudoku-data` (oder ein Bind-Mount) erhält den Spielstand zwischen Container-Neustarts. Das Marker-File `/var/opt/mssql/.killersudoku-schema-applied` verhindert mehrfaches Schema-Appl.

## CI / Registry-Push

Die GitHub-Actions-Pipeline `.github/workflows/docker.yml` baut bei jedem `push` auf `main` das Image und pusht es nach `ghcr.io/tim-fischer-zh/killer-sudoku` mit Tags:

- `:latest` (auf default-Branch)
- `:sha-<git-short-sha>` (immutable per Commit)
- `:<branch>` / `:<tag>` bei feature-Banches und Releases

Die Pipeline nutzt GHA-Cache ( `type=gha` ) für Schichten — der Multi-Stage-`dotnet publish` läuft inkremental und nicht bei jedem Push komplett neu.

## Submission-Inhalt (Update zu §7.4)

Zusätzlich zum ZIP-Inhalt aus §7.4 enthält das Repository (und damit `src/` im ZIP):

- `Dockerfile` (Repo-Root)
- `docker-entrypoint.sh` (Repo-Root)
- `docker-compose.yml` (Repo-Root)
- `.dockerignore` (Repo-Root)
- `.github/workflows/docker.yml`

Der Prüfer kann damit zwischen den drei Optionen (Pull / Local-Build / Compose) wählen.

---

## Verweise

- [Kapitel 2 — Randbedingungen](#)
- [Kapitel 5 — Bausteinsicht](#)
- [Kapitel 8 — Querschnittliche Konzepte](#)
- [Kapitel 9 — Architekturentscheidungen](#)
- **Funktionalitäts-Matrix** — Service-Layer-Inventar
- **Datenbank-Skript** — DB-Schema
- **Aufgabenstellung** — Aufgabenstellung



# 8 Querschnittliche Konzepte

arc42 v8.2 · Skills Battle 2026 — Application Development — Killer Sudoku Quelldokument  
(autoritativ): **Aufgabenstellung**

Dieses Kapitel sammelt Konzepte, die mehrere Bausteine oder Schichten der Architektur betreffen. Jedes Konzept verweist zurück auf seine UC- und Validation-Anker; eigentliche Regel-Listen werden nicht dupliziert, sondern referenziert.

## §8.1 Authentifizierung & Autorisierung

### Konzept

Die Anwendung verwendet **ASP.NET Core Identity** als Authentifizierungs-Provider und Cookie-basierte Sessions. Login ist Pflicht für alle nicht-öffentlichen Use Cases (UC04–UC14). UC01 (Read Rules) ist die einzige öffentliche Seite.

### Anforderungen aus README

- README §2.1 UC2: "To use the application, a login **is required**."
- README §2.1 UC3: "The user logs in."

### Implementations-Konzept

| Aspekt              | Lösung                                                                                                   |
|---------------------|----------------------------------------------------------------------------------------------------------|
| Identity-Provider   | ASP.NET Core Identity (Standard <code>IdentityUser</code> -Schema, lokale Tabelle <code>AppUser</code> ) |
| Passwort-Hashing    | ASP.NET Identity Default (PBKDF2 mit HMAC-SHA256, 100 000 Iterationen) — siehe <b>V03</b>                |
| Session-Mechanismus | Cookie (HttpOnly + Secure + SameSite=Lax — siehe §8.9)                                                   |
| Rate-Limiting Login | 5 Fehlversuche / 5 Minuten — siehe <b>V04</b>                                                            |
| Geschützte Routen   | <code>[Authorize]</code> -Attribut auf Razor-Pages bzw. Components — siehe <b>V16</b>                    |
| Cross-User-Mutation | Server prüft <code>game.UserId == currentUserId</code> vor jeder schreibenden Aktion — siehe <b>V16</b>  |

### Validation-Anker

- V03 — Passwort
- V04 — Login Rate-Limit
- V16 — Authorization

### Use-Case-Anker

- UC02 — Create User
- UC03 — Login

## §8.2 Eingabe-Validierung (Defense in Depth)

### Konzept

Alle User-Inputs werden auf **mindestens zwei Layern** geprüft:

1. **Client** (Blazor `EditForm` + Data Annotations + JS-Pattern)
2. **Server** (Service-Layer)
3. **Datenbank** (CHECK / UNIQUE / FK / Trigger — letzte Verteidigungslinie)

Diese Schichtung folgt der "Defense in Depth"-Konvention aus **Validation-Regeln**:

"Defense in Depth: Kritische Regeln werden auf mindestens 2 Layern geprüft. DB-Constraints sind die letzte Verteidigungslinie."

### Übersicht

Die vollständige Regel-Liste mit allen 16 Regeln (V01–V16) ist in **Validation-Regeln** dokumentiert. Hier nur die Layer-Zuordnungs-Heuristik:

| Regel-Kategorie                          | Client | Server                  | DB                      |
|------------------------------------------|--------|-------------------------|-------------------------|
| Format (Username, Email, Pattern)        | ✓      | ✓                       | partiell (UNIQUE, LIKE) |
| Range (Difficulty, CellValue, MarkValue) | ✓      | ✓                       | ✓ (CHECK)               |
| Solvability (UC05)                       | —      | ✓ (Solver)              | —                       |
| Auth & Authorization                     | —      | ✓                       | partiell (FK)           |
| XSS-Schutz                               | —      | ✓ (Razor-Auto-Encoding) | —                       |
| CSRF-Schutz                              | —      | ✓ (Antiforgery)         | —                       |

## Validation-Anker

Vollständige Liste aller Regeln: **Validation-Regeln** V01–V16.

## §8.3 Fehlerbehandlung

### Konzept

Fehler werden auf zwei semantische Klassen aufgeteilt:

1. **Erwartete Fachfehler** (z.B. Username vergeben, Puzzle unlösbar, falsche Credentials) → strukturierte `Result<T>` - bzw. Either-Typen
2. **Unerwartete System-Fehler** (DB-Down, NullRef, etc.) → ASP.NET-Exception-Middleware → generische Fehlerseite

### Strukturierte Result-Types

Service-Methoden geben statt Exceptions strukturierte Result-Typen zurück (siehe Service-Signaturen in **Funktionalitäts-Matrix**):

| Result-Type                   | Verwendet von                                      | Quelle |
|-------------------------------|----------------------------------------------------|--------|
| <code>RegisterResult</code>   | <code>IAuthService.RegisterAsync</code>            | UC02   |
| <code>LoginResult</code>      | <code>IAuthService.LoginAsync</code>               | UC03   |
| <code>ValidationResult</code> | <code>IPuzzleService.ValidateStructureAsync</code> | UC04   |
| <code>SavePuzzleResult</code> | <code>IPuzzleService.SaveIfSolvableAsync</code>    | UC05   |
| <code>CheckResult</code>      | <code>IGameService.CheckSolutionAsync</code>       | UC09   |
| <code>HintResult</code>       | <code>IHintService.GetHintAsync</code>             | UC07   |
| <code>SolveResult</code>      | <code>ISolverService.Solve</code>                  | UC11   |

### Generische Login-Fehlermeldung

Bei falschen Credentials liefert UC03 absichtlich **keine** Auskunft darüber, ob der Username existiert (Security-Pattern gegen Account-Enumeration):

"Username oder Passwort falsch."

Siehe **V03** und UC03 AC03.1:

"AC03.1: Falsche Credentials → keine Auskunft darüber, ob Username existiert"

### Use-Case-Anker

Alternative Pfade ( `Alt` ) sind pro UC in **Use-Cases-Dokument** dokumentiert.

## §8.4 Logging

### Konzept

Logging via `Microsoft.Extensions.Logging` (Standard-Abstraktion in .NET 10). Konkreter Provider: Console + File (Debug-Builds) / Console + EventLog (Release auf Windows).

### Was geloggt wird

| Ereignis                                                 | Log-Level   | Begründung                                               |
|----------------------------------------------------------|-------------|----------------------------------------------------------|
| Anwendungs-Start / Shutdown                              | Information | Operative Sichtbarkeit                                   |
| Fehlgeschlagene Login-Versuche                           | Warning     | Rate-Limit-Anker (siehe <b>V04</b> )                     |
| Solver-Aufrufe mit Dauer > 1s                            | Warning     | Performance-Boundary aus UC11 AC11.2 (" $< 2$ Sekunden") |
| Puzzle-Save (Solvability-Check Ergebnis: 0/1/ $\geq 2$ ) | Information | Audit für UC05                                           |
| Unbehandelte Exceptions                                  | Error       | Standard                                                 |
| HTTP-Requests (Method, Path, Status, Duration)           | Information | ASP.NET-Built-In Request-Logging                         |

### Was NICHT geloggt wird (kritisch)

- **Passwörter** — weder Klartext noch Hash, weder in Request-Bodies noch in Trace-Logs (siehe **V03**: "NIE im Klartext loggen/serialisieren")
- **Session-Cookies / Antiforgery-Token** — kein Logging von Request-Headers oberhalb von Debug-Level
- **PII über Username/Email hinaus** — UC erfordert keine weitergehenden Daten (siehe ERD in ER-Modell)

### Validation-Anker

- **V03**

## §8.5 Solver-Domänen-Konzept

### Konzept

Der Solver (UC11) ist die zentrale Logik-Komponente und das **erste Qualitätsziel** der Architektur (siehe [Kapitel 1 §1.2 Q1](#)). Er wird daher als **Pure Function** im Domain-Layer implementiert:

- **Keine** Abhängigkeit auf `DbContext` oder IO
- **Keine** Abhängigkeit auf ASP.NET-Infrastruktur
- Eingabe: `int[, ] givenValues` + `IReadOnlyList<CageDef> cages`
- Ausgabe: `SolveResult { Solutions, Solution? }`

### Algorithmik

Aus **UC11**:

"Solver implementiert Backtracking mit Constraint-Propagation:

- Standard-Sudoku-Constraints (Row/Col/Nonet)
- Killer-Cage-Constraints (Summe + keine Duplikate in Cage)"

Der Solver bricht nach der **zweiten** gefundenen Lösung ab — siehe [ADR-010](#).

### Aufrufer

| Aufrufer                                        | UC   | Verwendung                                                                           |
|-------------------------------------------------|------|--------------------------------------------------------------------------------------|
| <code>IPuzzleService.SaveIfSolvableAsync</code> | UC05 | <code>CountSolutions == 1</code> ⇒ Puzzle wird gespeichert                           |
| <code>IHintService.GetHintAsync</code>          | UC07 | Solver liefert Solution; Hint-Strategie wählt Zelle (siehe <a href="#">ADR-013</a> ) |
| <code>IGameService.CheckSolutionAsync</code>    | UC09 | Optional: Solver bestätigt User-Eingabe vs. eindeutige Lösung                        |

### Testbarkeit

Da der Solver keine externen Dependencies hat, ist er als **reiner Unit-Test** prüfbar:

- xUnit-Tests gegen die 2 README-Beispiele (siehe README §1.2: "You get 2 examples to test your application.")
- Edge-Cases: unlösbar, mehrdeutig, Single-Cell-Cage (Sum=N erzwingt Wert N)
- Performance-Boundary: < 2 Sekunden (UC11 AC11.2)

## Use-Case-Anker

- UC11 — Auto Solve
- [ADR-010](#)
- [ADR-013](#)

## §8.6 Datenzugriff

### Konzept

Datenzugriff über **Entity Framework Core 10** mit eigenem `SudokuDbContext`. Migrations werden **nicht** mit EF-Migrations gemacht, sondern das DB-Schema wird über ein deklaratives SQL-Skript (**Datenbank-Skript**) angelegt — das entspricht der README-Vorgabe (siehe README §1.3, Submission-Artefakt).

### DbContext-Scope

| Aspekt            | Lösung                                                                                                                            |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Lebenszyklus      | Scoped pro Blazor-Server-Circuit (eine Connection pro Tab/Session)                                                                |
| Connection-String | Aus <code>appsettings.json</code> (siehe <a href="#">Kapitel 7 §7.3</a> )                                                         |
| Tracking          | Default <code>AsTracking</code> für UPDATES (GameCell), <code>AsNoTracking</code> für Read-Only-Queries (Puzzle-Liste, Highscore) |
| Transaktionen     | Pro Service-Call eine Transaction wenn mehrere Tabellen geschrieben werden (z.B. UC05: Puzzle + Cage + CageCell)                  |

### Migrations

Migration-Strategie: **Single deklaratives SQL-Skript** statt EF-Migrations.

Begründung:

- README §1.3 fordert explizit ein **Datenbank-Skript**-Skript mit CREATE-Statements
- Idempotenz wird durch Drop-If-Exists am Skript-Anfang gewährleistet (siehe **Datenbank-Skript** Zeilen 22–31)
- Wettbewerbs-Submission soll für Prüfer 1-Click-reproduzierbar sein (SSMS + Run Script)

## EF-Modell-Mapping

| C#-Klasse               | Tabelle                 | Bemerkung                                                                                 |
|-------------------------|-------------------------|-------------------------------------------------------------------------------------------|
| <code>AppUser</code>    | <code>AppUser</code>    | Spalten-Name aus T-SQL-Reserved-Konflikt — siehe <a href="#">ADR-005</a>                  |
| <code>Puzzle</code>     | <code>Puzzle</code>     | KEINE Solution-Spalte (siehe <a href="#">ADR-007</a> )                                    |
| <code>Cage</code>       | <code>Cage</code>       | mit <code>[Sum]</code> -Property (T-SQL-Reserved)                                         |
| <code>CageCell</code>   | <code>CageCell</code>   | <code>RowIdx / ColIdx</code> statt <code>Row / Col</code> — siehe <a href="#">ADR-006</a> |
| <code>Game</code>       | <code>Game</code>       | Pause-Felder siehe <a href="#">ADR-009</a>                                                |
| <code>GameCell</code>   | <code>GameCell</code>   | <code>CellValue</code> NULL-fähig                                                         |
| <code>PencilMark</code> | <code>PencilMark</code> | eigene Tabelle, kein JSON — siehe <a href="#">ADR-008</a>                                 |
| <code>HintLog</code>    | <code>HintLog</code>    | Audit-Log                                                                                 |

## Highscore-View

Highscore ist eine **DB-View** `vw_Highscore` (siehe **Datenbank-Skript** und [ADR-003](#)). EF mappt sie als read-only Keyless-Entity. Der Service `IHighscoreService.GetTopAsync` selektiert nur aus dieser View und sortiert nach `Score DESC`.

## Anker

- **ER-Modell** — vollständiges ERD mit Spalten-Typen
- **Datenbank-Skript** — Schema
- **Funktionalitäts-Matrix** — Service × DB-Operations-Matrix

## §8.7 UI-Komponenten

### Konzept

Die UI ist in einer **Blazor-Component-Hierarchie** organisiert. Pro Screen (S1–S7) existiert eine Root-Component ( `@page` -Route), die kleinere Children komponiert. Komponenten sind nach **Single Responsibility** geschnitten und einzeln via bUnit testbar.

### Component-Hierarchie

Übersicht aus **Funktionalitäts-Matrix** (Spalte "Blazor-Component"):

```

Layout.razor
├── Home.razor (S1, public, UC01)
│   ├── RulesPanel.razor
│   └── MiniSudokuExample.razor
├── Register.razor (S2, public, UC02)
│   └── RegisterForm.razor
├── Login.razor (S3, public, UC03)
│   └── LoginForm.razor
├── PuzzleList.razor (S4, [Authorize], UC12)
│   ├── FilterBar.razor
│   └── PuzzleCard.razor
├── EnterPuzzle.razor (S5, [Authorize], UC04+UC05)
│   ├── PuzzleGrid.razor
│   └── CageEditor.razor
├── PlayPuzzle.razor (S6, [Authorize], UC06–UC09, UC13, UC14)
│   ├── PuzzleGrid.razor
│   │   └── PencilMarkLayer.razor (UC14)
│   ├── Toolbar.razor
│   │   ├── HintButton.razor (UC07)
│   │   ├── PauseButton.razor (UC13)
│   │   └── CheckSolutionButton.razor (UC09)
│   └── Highscore.razor (S7, [Authorize], UC08)
│       └── HighscoreTable.razor

```

## Quell-Anker

- Screen-Inventar: **Funktionalitäts-Matrix**
- Component × UC × Service × DB: **Funktionalitäts-Matrix**
- Visuelle Briefings: **Mockup-Briefings**

## Component-Kommunikation

| Pattern                  | Verwendung                                       |
|--------------------------|--------------------------------------------------|
| [Parameter]              | Eltern → Kind (Down)                             |
| EventCallback<T>         | Kind → Eltern (Up)                               |
| CascadingValue           | Spielzustand in <code>PlayPuzzle</code> -Subtree |
| Service-Injection via DI | Components → Service-Layer                       |

## State-Management

Spielzustand (Grid, Pencil Marks, Pause-Status) lebt auf dem Server pro Blazor-Circuit. Bei Disconnect/Reconnect lädt der Component-Init aus `IGameService` den persistierten Zustand aus der DB nach (Auto-Save bei jedem Move — siehe **UC13** "Browser-Tab schliessen ohne Pause → State wird trotzdem persistiert").



## §8.8 Internationalisierung

### Konzept

**UI-Sprache: Deutsch** (einsprachig). Es ist keine Internationalisierung (i18n) implementiert.

### Begründung

- Das README spezifiziert keine i18n-Pflicht (kein "must" oder "required" zu Sprachen)
- Der Wettbewerb ist im deutschsprachigen Raum verortet (Skills Battle)
- Die Stakeholder-Tabelle in [Kapitel 1 §1.3](#) hält fest: "UI-Sprache deutsch."

### Umsetzung

- Alle UI-Strings sind deutsch hardkodiert in Razor-Markup oder als `const string` (kein `IStringLocalizer`)
- Fehlermeldungen sind deutsch (Beispiele aus **Validation-Regeln**):
- "Schwierigkeit muss 1, 2 oder 3 sein."
- "Username oder Passwort falsch."
- "Puzzle ist nicht lösbar."
- Datums-/Zeit-Formate: `de-CH` Culture (kann via `appsettings.json` umgestellt werden, ist aber kein Pflicht-Feature)

Falls i18n nachträglich gefordert wird, ist die Migration unproblematisch ( `.resx` -Files + `@inject IStringLocalizer` ). Aktuell aus Scope ausgeschlossen.

## §8.9 Sicherheit

### Konzept

Sicherheit ist als **Defense in Depth** über mehrere Layer organisiert. Die hier aufgeführten Massnahmen leiten sich direkt aus den README-Vorgaben (Login Pflicht, Solution-Eindeutigkeit) und den Validation-Regeln V01–V16 ab.

## Massnahmen-Matrix

| Bedrohung                                     | Massnahme                                                                                                               | Layer                           | Anker                      |
|-----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|---------------------------------|----------------------------|
| Account-Diebstahl (Replay/Theft)              | Auth-Cookie: <code>HttpOnly</code> + <code>Secure</code> + <code>SameSite=Lax</code>                                    | Server (Identity-Konfiguration) | <b>V15</b> , UC03 AC03.3   |
| Cross-Site Request Forgery                    | ASP.NET Antiforgery Token (Standard in Blazor Server <code>EditForm</code> )                                            | Server                          | <b>V15</b>                 |
| Cross-Site Scripting (XSS)                    | Razor <code>@expression</code> → automatisches HTML-Encoding; <b>kein</b> <code>@(MarkupString)</code> für User-Content | Server                          | <b>V14</b>                 |
| SQL-Injection                                 | EF Core Parameterized Queries (keine String-Konkatenation in SQL)                                                       | Server                          | implicit (EF Core Default) |
| Passwort-Klartext-Leak                        | PBKDF2-Hash via ASP.NET Identity; <b>NIE</b> Klartext loggen                                                            | Server                          | <b>V03</b> , §8.4          |
| Account-Enumeration                           | Generische Login-Fehlermeldung ("Username oder Passwort falsch")                                                        | Server                          | <b>V03</b> , UC03 AC03.1   |
| Brute-Force-Login                             | Rate-Limit: 5 Versuche / 5 Minuten                                                                                      | Server                          | <b>V04</b>                 |
| Cross-User-Mutation                           | Server-side Check <code>game.UserId == currentUserId</code> vor jedem Write                                             | Server                          | <b>V16</b>                 |
| Unauthorized Routes                           | <code>[Authorize]</code> -Attribut + Redirect zu <code>/login</code>                                                    | Server                          | <b>V16</b>                 |
| Puzzle-Manipulation (Hash-Probleme bei Solve) | Solver-Logik ist serverseitig, kein Trust auf Client-Input                                                              | Server                          | <b>UC09</b>                |

## Cookie-Konfiguration

```
services.ConfigureApplicationCookie(options =>
{
    options.Cookie.HttpOnly = true;
    options.Cookie.SecurePolicy = CookieSecurePolicy.Always;
    options.Cookie.SameSite = SameSiteMode.Lax;
    options.ExpireTimeSpan = TimeSpan.FromHours(24);
    options.SlidingExpiration = true;
});
```

**SameSite=Lax** (statt `Strict`) erlaubt Top-Level-Navigation mit Cookie (z.B. Klick auf externen Link, der zurück führt). `Lax` ist OWASP-Recommendation für Web-Apps mit

normalem Login-Flow.

## Verweise

- **Use-Cases-Dokument** — UC01–UC14
- **Validation-Regeln** — V01–V16
- **Funktionalitäts-Matrix** — Service- & UI-Inventar
- **ER-Modell** — DB-Modell & Design-Entscheidungen
- [Kapitel 7 — Verteilungssicht](#)
- [Kapitel 9 — Architekturentscheidungen](#)

## 9 Architekturentscheidungen (ADRs)

arc42 v8.2 · Skills Battle 2026 — Application Development — Killer Sudoku Quelldokument  
(autoritativ): **Aufgabenstellung**

Dieses Kapitel dokumentiert die wesentlichen Architekturentscheidungen als ADRs (Architecture Decision Records). Jeder Eintrag folgt dem Format:

- **Titel** — Was wurde entschieden
- **Status** — `Akzeptiert` (alle hier dokumentierten ADRs sind angenommen)
- **Kontext** — Welches Problem / welche Anforderung führt zur Entscheidung
- **Entscheidung** — Was wurde konkret gewählt
- **Konsequenz** — Welche positiven & negativen Folgen ergeben sich

### ADR-001 — Stack: .NET 10 + Blazor Server + MS-SQL Server

**Status:** Akzeptiert

**Kontext:** Die Aufgabenstellung (Skills Battle 2026 — Skill 7) erlaubt verschiedene Implementations-Stacks (PHP, Node.js, .NET). README §1.3 fordert lediglich: "create a database named 'sudoku' on **MySQL or MS-SQL** server running on your machine." Die UI-Technologie ist frei wählbar.

**Entscheidung:** Stack: **.NET 10** (Blazor Server) als Anwendungs-Framework und **MS-SQL Server Express** als Datenbank.

**Konsequenz:**

- (+) Strenges Typsystem (C# 13) reduziert Fehler bei Solver- und Domain-Logik
- (+) Blazor Server: vollständiges C#-Stack (Frontend + Backend), keine Sprach-Wechsel zwischen Layern
- (+) ASP.NET Core Identity bietet hardened-Auth out-of-the-box (siehe §8.1)
- (+) EF Core 10 + MS-SQL Express: konsistentes Tooling, Trusted Connection für lokales Setup
- (+) bUnit-Component-Tests + xUnit Unit-Tests sind im selben Test-Projekt möglich
- (–) Blazor Server benötigt permanente SignalR-Verbindung (siehe §7.1) — bei Disconnect Verlust des UI-State (mitigiert via Auto-Save in DB, **UC13**)
- (–) .NET-Setup auf Prüfer-Laptop muss vorhanden sein (Mitigation: Submission liefert Build-Output in `bin/Release/net10.0/`, siehe §7.4)

## ADR-002 — Cage-Modellierung relational (Cage + CageCell)

**Status:** Akzeptiert

**Kontext:** Ein Killer-Sudoku besteht aus Cages: Gruppen zusammenhängender Zellen mit einer Soll-Summe. Pro Puzzle gibt es 1–N Cages, jeder Cage hat 1–9 Zellen. Eine Zelle gehört genau einem Cage an. Modellierungs-Optionen:

1. **Relational** — Tabelle `Cage` (Id, PuzzleId, Sum) + Junction-Tabelle `CageCell` (CageId, RowIdx, ColIdx)
2. **JSON in Puzzle** — Eine Spalte `CagesJson NVARCHAR(MAX)` mit Array von Cages

**Entscheidung: Variante 1 (relational).** Siehe Schema in **Datenbank-Skript** (Zeilen 67–96).

**Konsequenz:**

- (+) FK-Integrität & DB-CHECK-Constraints ( $\text{Sum} \in [1, 45]$ ,  $\text{Row/Col} \in [0, 8]$ ) — siehe **V06**
- (+) Einfache SQL-Queries für Cage-Sum-Validation (UC09) — siehe Sample-Queries in **ER-Modell**
- (+) Indexierbarkeit (Index `IX_Cage_PuzzleId`)
- (+) Trigger `trg_CageCell_UniquePerPuzzle` lässt sich nur in der relationalen Variante sinnvoll umsetzen
- (–) Mehr Joins beim Lesen eines Puzzles (zwei Queries: Cages + CageCells, oder ein Join)
- (–) Speicher-Overhead minimal (TINYINT-Spalten + Composite PKs)

## ADR-003 — Highscore als View statt Tabelle

**Status:** Akzeptiert

**Kontext:** UC08 (Show High Score) zeigt die Top-N abgeschlossenen Games sortiert nach Score. Optionen:

1. **Denormalisierte Tabelle** `Highscore` mit Sync-Logik bei UC10 (Save Result)
2. **View** `vw_Highscore` über `Game JOIN AppUser JOIN Puzzle`

**Entscheidung: Variante 2 (View).** Siehe **Datenbank-Skript** Zeilen 214–232.

**Konsequenz:**

- (+) Single Source of Truth: kein Sync-Risiko zwischen `Game` und `Highscore`
- (+) Automatisch konsistent bei UPDATE auf `Game`
- (+) `WHERE g.IsCompleted = 1 AND g.Score IS NOT NULL` stellt sicher, dass nur abgeschlossene Spiele erscheinen — passt zu UC10 AC10.3
- (–) Performance: bei sehr vielen Games ( $>10^6$ ) wäre View langsamer als materialisierte Tabelle. Für die Wettbewerbs-Submission (lokales Setup, wenige Spiele) irrelevant.

- (–) Pagination der View erfordert `OFFSET/FETCH` in jeder Abfrage statt vorberechneter Sortierung

## ADR-004 — "Eine Zelle pro Cage pro Puzzle" via Trigger

**Status:** Akzeptiert

**Kontext:** Die Regel "jede Zelle gehört zu **genau einem** Cage pro Puzzle" (siehe UC04 AC04.2 und **V06**) lässt sich auf Datenbank-Ebene nicht direkt mit einer einzelnen UNIQUE-Constraint ausdrücken, weil `CageCell` kein direktes `PuzzleId`-Feld hat (PuzzleId steckt in `Cage`).

Optionen:

1. **Composite FK über (PuzzleId, RowIdx, ColIdx)** — würde `PuzzleId` zusätzlich in `CageCell` duplizieren
2. **AFTER INSERT/UPDATE Trigger** — prüft Cross-Cage-Konflikt im Puzzle
3. **Nur Application-Layer-Validierung** — kein DB-Schutz

**Entscheidung: Variante 2 (Trigger).** Konkret `trg_CageCell_UniquePerPuzzle` (siehe **Datenbank-Skript** Zeilen 100–121):

```
CREATE OR ALTER TRIGGER trg_CageCell_UniquePerPuzzle
ON CageCell
AFTER INSERT, UPDATE
AS
BEGIN
    ...
    IF EXISTS (
        SELECT 1
        FROM CageCell cc
        JOIN Cage c1 ON c1.Id = cc.CageId
        JOIN inserted i ON i.RowIdx = cc.RowIdx AND i.ColIdx = cc.ColIdx
        JOIN Cage c2 ON c2.Id = i.CageId
        WHERE c1.PuzzleId = c2.PuzzleId
        AND cc.CageId <> i.CageId
    )
    BEGIN
        ROLLBACK TRANSACTION;
        THROW 50001, 'CageCell: Cell already assigned to another cage in the same puzzle.', 1
    END
END;
```

**Konsequenz:**

- (+) Defense-in-Depth: Application-Validierung (V06) + DB-Trigger
- (+) Schema-Normalisierung erhalten — kein redundantes `PuzzleId` in `CageCell`
- (+) Klare Fehlermeldung im Konflikt-Fall ( `THROW 50001` )

- (–) Trigger-Logik ist weniger sichtbar als deklarative Constraints
- (–) Performance-Overhead bei Bulk-Inserts (eher irrelevant: 81 Zellen pro Puzzle)
- Begründung im Kommentar des SQL-Skripts (Zeile 84–86): "Constraint 'jede Zelle pro Puzzle in genau einem Cage' wird über einen INSTEAD-OF-Trigger durchgesetzt"

**Anmerkung zum SQL-Kommentar:** Der Kommentar im Skript spricht von "INSTEAD-OF-Trigger", umgesetzt wird tatsächlich ein `AFTER INSERT, UPDATE` Trigger. Funktional identisch (Rollback bei Konflikt), aber Wording im Kommentar bleibt zur Konsistenz mit dem Submission-File.

## ADR-005 — AppUser statt User (T-SQL Reserved Keyword)

**Status:** Akzeptiert

**Kontext:** Die Benutzer-Tabelle würde idiomatisch `User` heissen. In T-SQL ist `USER` jedoch ein reserviertes Keyword (für `USER_NAME` etc.), was sowohl im Schema-Skript als auch in EF-LINQ-Queries ständige Eckige-Klammern-Quotation erfordern würde.

**Entscheidung:** Tabelle und C#-Entity heissen `AppUser`. Siehe **ER-Modell** und **Datenbank-Skript** Zeilen 36–48.

**Konsequenz:**

- (+) Kein Quoting notwendig, klare Lesbarkeit in SQL und C#
- (+) Konvention findet sich in vielen ASP.NET-Identity-Templates (`AspNetUsers`, `ApplicationUser` etc.)
- (–) Leichte Reibung mit dem Use-Case-Vokabular ("User" als Begriff bleibt im Sprachgebrauch)

## ADR-006 — RowIdx / ColIdx statt Row / Col

**Status:** Akzeptiert

**Kontext:** Sudoku-Zellen werden durch Zeilen- und Spalten-Index identifiziert. Idiomatisch wären die Spalten-Namen `Row` und `Col`. T-SQL reserviert allerdings `ROW` (z.B. in `ROW_NUMBER` und Table-Hints).

**Entscheidung:** Spalten heissen `RowIdx` und `ColIdx` (TINYINT, Range 0–8). Siehe **ER-Modell** und **Datenbank-Skript**.

**Konsequenz:**

- (+) Kein Quoting in SQL

- (+) Suffix `Idx` macht den Zero-Based-Index explizit (nicht 1-Based wie in der UI)
- (–) Etwas länger als `Row / Col` — minimaler Cosmetic-Impact

## ADR-007 — KEINE Solution-Spalte in Puzzle

**Status:** Akzeptiert

**Kontext:** Ein Puzzle hat genau eine eindeutige Lösung (gefordert durch README §1 "The solution **must be unique**." und durchgesetzt durch UC05). Naheliegende Optimierung wäre, die berechnete Lösung beim Save zu cachen.

Die Aufgabenstellung verbietet das jedoch explizit. README §2.1 UC4 wörtlich:

"**No solution is recorded.** Solutions must be calculated with an algorithm."

**Entscheidung:** Die Tabelle `Puzzle` enthält **keine** Solution-Spalte. Lösungen werden bei jedem Bedarf (UC05, UC07, UC09) on-the-fly via `ISolverService.Solve` berechnet.

**Konsequenz:**

- (+) Konform zur strikten README-Vorgabe — bestehensrelevant
- (+) Single Source of Truth: Solver ist die Autorität, kein DB-Cache kann veralten
- (+) Test: SQL-Query auf `INFORMATION_SCHEMA.COLUMNS` zeigt keine `Solution`-Spalte (UC04 AC04.4)
- (–) Solver wird mehrfach aufgerufen pro Game-Session (z.B. mehrere Hints) — mitigiert durch In-Memory-Cache pro Circuit, **niemals** persistent
- Konsequenz aus Strict-Word-Audit S4 in **Use-Cases-Dokument**

## ADR-008 — Pencil Marks als eigene Tabelle (statt JSON in GameCell)

**Status:** Akzeptiert

**Kontext:** UC14 (Pencil Marks) erlaubt 0–9 Kandidaten-Markierungen pro Zelle. Modellierungs-Optionen:

1. **Eigene Tabelle** `PencilMark` mit Composite PK (`GameId`, `RowIdx`, `ColIdx`, `MarkValue`)
2. **JSON-Spalte** in `GameCell` (z.B. `PencilMarksJson NVARCHAR(50)`)
3. **Bitmaske** in `GameCell` (z.B. `TINYINT PencilBitmask` — Bits 0–8 für Zahlen 1–9)

**Entscheidung: Variante 1 (eigene Tabelle).** Siehe **ER-Modell** und **Datenbank-Skript** Zeilen 180–192.



**Konsequenz:**

- (+) Konsistenz mit relationaler Cage-Modellierung (ADR-002)
- (+) CHECK-Constraint `CK_PencilMark_Value` (Range 1–9) auf DB-Ebene
- (+) Composite PK verhindert Duplikate (siehe **V13**)
- (+) Indexier-/Query-bar (z.B. "alle Marks für Gameld X")
- (–) Mehr Rows in DB (worst case  $81 \times 9 = 729$  Marks pro Game) — bei lokalem Setup unkritisch
- (–) JSON wäre kompakter, aber kein Schutz vor Invariant-Verletzungen

**ADR-009 — Pause via TotalPausedSeconds + PausedAt****Status:** Akzeptiert

**Kontext:** UC13 (Pause & Resume) erfordert, dass die gespielte Netto-Zeit korrekt berechnet wird, auch wenn der User mehrmals pausiert. Optionen:

1. **Multi-Row Pause-Log** — Separate Tabelle `PauseEvent` (`GameId`, `PausedAt`, `ResumedAt`), Summe aller Differenzen ist die Pause-Zeit
2. **Single Row mit Accumulator** — Felder `IsPaused`, `PausedAt`, `TotalPausedSeconds` direkt in `Game`
3. **Nur StartTime adjustieren** — Bei Resume die `StartTime` nach hinten schieben

**Entscheidung: Variante 2.** `Game` hat drei Pause-Felder:

- `IsPaused BIT` — aktuell pausiert ja/nein
- `PausedAt DATETIME2 NULL` — Zeitpunkt der aktuellen Pause
- `TotalPausedSeconds INT` — kumulierte Pause-Zeit über alle bisherigen Intervalle

Berechnung beim Game-Ende:

```
TimeSeconds = DATEDIFF(SECOND, StartTime, EndTime) - TotalPausedSeconds
```

Siehe **Datenbank-Skript** Zeilen 128–149 und **ER-Modell**.

**Konsequenz:**

- (+) Einfach zu queryen, kein Aggregat über Sub-Tabelle nötig
- (+) Audit-Spur durch `PausedAt` (letzter Pause-Beginn) — operative Sichtbarkeit
- (+) Konsistenz-Check `CK_Game_TotalPaused CHECK (TotalPausedSeconds >= 0)` (**V12**)
- (–) Keine Historie aller Pause-Events (z.B. "wie oft wurde pausiert"). Aus README nicht gefordert.
- (–) Resume-Logik muss atomic sein (read `PausedAt`, compute  $\Delta$ , add to `TotalPausedSeconds`, set `IsPaused=0`, `PausedAt=NULL`) — wird via Service-Transaction abgesichert

## ADR-010 — Solver-Output: Limit auf 2 Lösungen

**Status:** Akzeptiert

**Kontext:** Der Solver muss drei Fälle unterscheiden (siehe **V07**):

- 0 Lösungen → "Puzzle ist nicht lösbar"
- 1 Lösung → speichern (UC05) bzw. Hint geben (UC07)
- $\geq 2$  Lösungen → "Lösung muss eindeutig sein"

Wenn der Solver mehrere Lösungen finden würde, könnte er stundenlang laufen ( $10^9$ + Möglichkeiten bei wenig vorgegebenen Zellen).

**Entscheidung:** Der Solver-Algorithmus bricht **nach der zweiten** gefundenen Lösung ab.  
Rückgabe:

```
public record SolveResult(int Solutions, int[,]? Solution);  
// Solutions ∈ {0, 1, 2}  
// 2 bedeutet: "mindestens 2 gefunden, weitere möglich"
```

Siehe UC11 Main Flow Schritt 5:

"Bei Solutions  $\geq 2$  → nicht eindeutig (Solver bricht nach 2. gefundener Lösung ab)"

**Konsequenz:**

- (+) Performance: Worst-Case bounded, keine Abbruch-Heuristik nötig
- (+) Genügt für UC05-Entscheidung (Save ja/nein) und UC11-Aussage (Eindeutigkeit ja/nein)
- (+) Boundary-Test: UC11 AC11.2 " $< 2$  Sekunden"
- (–) Keine genaue Anzahl Lösungen bei ambiguous Puzzles — aber: nicht gefordert
- (–) Hint-Algorithmus muss bei `Solutions != 1` graceful failen (Hint nur bei eindeutiger Lösung sinnvoll)

## ADR-011 — Test-Pyramide: xUnit + bUnit + WebApplicationFactory + Playwright .NET

**Status:** Akzeptiert

**Kontext:** README §3.2 fordert wörtlich:

"Running the test cases should be implemented with a **test framework** and is part of the submission (e.g. JUnit in Java)."

Plus §3.1 (mind. 1 positiver, 1 negativer, ggf. 1 Boundary-Test pro UC).

Test-Pyramide-Optionen für den .NET-Stack:

| Layer             | Framework-Optionen                                              |
|-------------------|-----------------------------------------------------------------|
| Unit              | xUnit, NUnit, MSTest                                            |
| Component (Razor) | bUnit, AngleSharp-DOM-Setup                                     |
| Integration       | xUnit + <code>WebApplicationFactory&lt;T&gt;</code> (offiziell) |
| E2E               | Playwright .NET, Selenium, Cypress                              |

#### Entscheidung:

- **Unit:** xUnit (Solver, Service-Methoden, Validation-Regeln)
- **Component:** bUnit (Razor-Components — Render, Event, State)
- **Integration:** xUnit + `WebApplicationFactory` (echte DB-Roundtrips, Auth-Flow)
- **E2E:** Playwright .NET (kritische User-Flows: Register → Login → Solve)

#### Konsequenz:

- (+) Alle Layer als `dotnet test` aufrufbar — eine CI-Pipeline
- (+) bUnit ist der De-facto-Standard für Blazor-Components (Microsoft-empfohlen)
- (+) Playwright .NET liefert auch UI-Test-Recording (Trace-Files)
- (+) Solver-Tests als reine Unit-Tests möglich, da Solver Pure Function (§8.5)
- (–) Vier Frameworks im Stack — Lernkurve für Prüfer, der das Test-Protokoll liest. Mitigation: Test-Protokoll listet Framework pro Test-Case in der CSV (**Test-Protokoll**).
- (–) Playwright benötigt Browser-Binary-Download beim ersten Run

## ADR-012 — Score-Formel: $\max(0, 10000 - \text{TimeSeconds} - \text{HintsUsed} \times 300)$

**Status:** Akzeptiert

**Kontext:** README §2.1 UC8 fordert: "Implement rules for a high score based on the time needed and the number of hints used." Die konkrete Formel ist frei wählbar.

Anforderungen an die Formel:

- Schnellere Spiele → höherer Score

- Mehr Hints → niedrigerer Score (lt. UC07: "should be considered for the high score")
- Keine negativen Scores
- Ganzzahlig

**Entscheidung:**

$$\text{Score} = \max(0, 10000 - \text{TimeSeconds} - \text{HintsUsed} \times 300)$$

Siehe UC08 Score-Formel:

- Begründung: Zeit-Penalty linear (1 Pkt/Sek); Hints kosten 5 Min Äquivalent
- Floor bei 0 → keine negativen Scores

**Konsequenz:**

- (+) Einfach nachvollziehbar — Prüfer kann manuell prüfen
- (+) 5-Min-Penalty pro Hint ist eine "schmerzhaft" aber nicht ruinöse Wertung
- (+) `max(0, ...)` verhindert negative Scores → DB-Constraint `CK_Game_Score CHECK (Score >= 0)` erfüllt (**V12**)
- (–) Theoretisch könnte ein User in 9 999 Sekunden ohne Hints noch 1 Pkt erreichen — aber: nicht negativ, akzeptabel
- Test-Anker: **AC08.1** (Score ganzzahlig nach Formel)

## ADR-013 — Hint-Strategie: Naked-Single → Cage-Forced → Solver-Fallback

**Status:** Akzeptiert

**Kontext:** UC07 (Ask for a Hint) verlangt einen Hint, der dem Spieler hilft. README:

"Develop a **suggestion** for the hint, document it, and implement your solution."

Es gibt mehrere Hint-Qualitäten:

1. **Naked Single** — Zelle, in der nur 1 Wert constraint-konform möglich ist ("Logikhint")
2. **Cage-Forced** — Cage, dessen Belegung eindeutig durch die Summe + Constraints forciert ist
3. **Solver-Fallback** — wenn kein Logik-Hint möglich: einfach eine korrekte leere Zelle füllen

**Entscheidung:** Hint-Algorithmus versucht in dieser Reihenfolge:

1. **Strategy A — Naked Single:** Suche eine leere Zelle, deren mögliche Werte nach Anwendung aller Constraints (Row/Col/Nonet/Cage) genau eine Zahl ergibt. Wenn gefunden: Hint enthält "Hier kann nur N stehen".
2. **Strategy B — Cage-Forced:** Wenn keine Naked-Single existiert, suche einen Cage, dessen Lösung eindeutig ist (z.B. 2-Zellen-Cage mit Sum=17 in unterschiedlichen Rows → muss {8,9} sein).
3. **Fallback:** Wenn beide Strategien scheitern (selten bei harten Puzzles), lass den Solver eine beliebige leere Zelle der bekannten Lösung füllen.

Siehe UC07 Hint-Vorschlag (dokumentiert).

#### Konsequenz:

- (+) Pädagogischer Mehrwert: Hint erklärt die Logik (Strategy A/B), nicht nur das Ergebnis
- (+) Solver wird nur als Fallback gebraucht → Performance besser bei einfachen Puzzles
- (+) HintsUsed-Counter wird in jedem Fall inkrementiert (siehe UC07 Main)
- (–) Hint-Algorithmus ist komplexer als ein reiner Solver-Aufruf — mehr Code, mehr Tests
- (–) Tests müssen alle drei Pfade abdecken (Strategy A, B, Fallback)
- Test-Anker: **AC07.1** (Hint füllt nur korrekte Zelle)

## ADR-014 — Auswahl der 3 zusätzlichen Use Cases

**Status:** Akzeptiert

**Kontext:** README §2.3 fordert:

"Add additional use cases to the use case diagram. Think of **3 additional** use cases that you can implement. Choose them based on the existing entries. Write down your decision in the documentation."

Kandidaten für zusätzliche UCs:

- Browse / Filter Puzzles
- Pause / Resume Game
- Pencil Marks (Candidate Notes)
- Edit Profile / Change Password
- Delete eigenes Puzzle
- Statistik-Dashboard
- Multiplayer / Live-Wettkampf
- Daily-Challenge

**Entscheidung:** Gewählte UCs (siehe **Use-Cases-Dokument**):

- **UC12 — Browse / Filter Puzzles** — Praktischer UX-Mehrwert, DB-Query-relevant (Integration-Test-Anker)
- **UC13 — Pause / Resume Game** — Reale User-Anforderung (Sudoku-Spiele dauern lange), persistiert Game-State (Component- und Integration-Test-Anker)
- **UC14 — Pencil Marks** — Standard-Feature aller Sudoku-Apps, demonstriert UI-State-Management (Component-Test-Anker)

#### Begründung der Auswahl (wörtlich aus **Use-Cases-Dokument**):

- UC12 Browse/Filter — Praktischer Mehrwert (UX), DB-Query-relevant (Integration-Tests)
- UC13 Pause/Resume — Reale User-Anforderung (Sudoku-Spiele dauern lange), persistiert Game-State (Component + Integration)
- UC14 Pencil Marks — Standard-Feature aller Sudoku-Apps, demonstriert UI-State-Management (Component-Test)

→ Die 3 UCs decken bewusst 3 verschiedene Test-Schwerpunkte ab.

#### Konsequenz:

- (+) Jeder UC zielt auf eine andere Test-Schicht (DB-Integration / Game-State / UI-Component) — gute Demonstration der Test-Pyramide aus [ADR-011](#)
- (+) Jeder UC ergänzt Kernfunktionen (Solve-Flow), ohne neue Personas einzuführen → Scope bleibt eng
- (+) Pencil Marks und Pause sind Standard-Feature-Erwartungen → User-Experience-Boost
- (–) Bewusst NICHT gewählt: Multiplayer (würde Auth- und Sync-Komplexität explodieren), Admin (zweite Persona, mehr Routen)
- (–) Browse-Filter erfordert Pagination-Logik (siehe UC12 AC12.2)

## ADR-015 — Single-Image Container für Prüferbequemlichkeit

**Status:** Akzeptiert

**Kontext:** Die README-Aufgabenstellung verlangt explizit: "create a database named 'sudoku' on MySQL or MS-SQL server **running on your machine**" (§1.3). Das bedeutet, jeder Prüfer muss lokal eine MS-SQL-Instanz installieren, das Schema deployen, die App konfigurieren und starten. Diese Reibung ist für die Bewertung nicht förderlich.

Alternativen:

1. **Native-Only (Standard).** Prüfer installiert MS-SQL Express + .NET 10 SDK manuell, führt **Datenbank-Skript** aus, startet App via `dotnet run`. Standard und gut dokumentiert (siehe [§7.2](#) / [§7.3](#)) — aber: Setup-Reibung, Plattform-Drift macOS/Windows/Linux.

2. **Klassisches Docker-Compose mit 2 Services.** Ein `mssql`-Service + ein `app`-Service mit Network-Bridge. Best-Practice für Production (saubere Trennung, Skalierbarkeit) — aber: zwei Container, zwei Lifecycle-Probleme, Wait-for-DB-Race-Condition vor App-Start.
3. **Single-Image (DB + App in einem Container).** Multi-Stage Build mit `mssql/server:2022-latest` als Base + .NET-Runtime als Layer + Supervisor-Script ( `docker-entrypoint.sh` ) für sqlservr + dotnet. Anti-Pattern für Production, aber **ein einziger `docker run`-Befehl** für den Prüfer.

**Entscheidung: Option 3 (Single-Image)** wird als primärer Container-Pfad ausgeliefert, parallel zur nativen Variante.

#### Begründung:

- Der Wettbewerbs-Kontext ist Submission, nicht Production. Der Prüfer bewertet ein Mal — Skalierung oder Update-Pfade sind nicht relevant.
- README §1.3 ("running on your machine") wird trotz Container erfüllt — der Container läuft auf der Maschine des Prüfers, die DB läuft im Container des Prüfers. Kein Cloud-Service involviert.
- Das Image wird bei jedem `main`-Push via [.github/workflows/docker.yml](#) gebaut und nach `ghcr.io/tim-fischer-zh/killer-sudoku:latest` gepusht → Prüfer kann mit einem `docker pull` ohne Local-Build verifizieren.
- Supervisor-Script handhabt Race-Condition (Wait-for-DB), idempotenter Schema-Apply via Marker-File.
- Die Native-Variante bleibt als Fallback dokumentiert für Prüfer ohne Docker.

#### Konsequenz:

- (+) `docker run -p 8080:8080 ghcr.io/tim-fischer-zh/killer-sudoku:latest` und Browser → fertig.
- (+) Reproduzierbarkeit garantiert (Image-SHA in GHA-Tags).
- (+) MS-SQL EULA wird im Image akzeptiert (via `ACCEPT_EULA=Y`) — kein manuelles Setup.
- (–) Anti-Pattern für Production: zwei Prozesse in einem Container (Single-Process-Per-Container ist Best Practice).
- (–) Bigger image footprint (~ 1.5 GB inkl. mssql-Layer).
- (–) Logs der zwei Prozesse vermischen sich (stdout). Akzeptabel für Submission-Kontext.
- (–) Healthcheck deckt nur die App ab ( `curl :8080/health` ), nicht die DB direkt — DB-Health wird implizit über App-Health geprüft.

#### Operatives Setup:

- `Dockerfile`, `docker-entrypoint.sh`, `docker-compose.yml`, `.dockerignore` im Repo-Root
- GitHub-Actions-Workflow für Build & Push nach `ghcr.io`
- Persistenz via Named-Volume `mssql-data`
- Connection-String über `ConnectionStrings__Sudoku` Env-Var override-bar

→ Vollständiges Setup in [§7.5 Container-Deployment](#).

## Übersicht aller ADRs

| ADR                     | Titel                                   | Bezug                                   |
|-------------------------|-----------------------------------------|-----------------------------------------|
| <a href="#">ADR-001</a> | Stack: .NET 10 + Blazor Server + MS-SQL | <a href="#">§7</a> , <a href="#">§8</a> |
| <a href="#">ADR-002</a> | Cage relational modelliert              | <b>ER-Modell, Datenbank-Skript</b>      |
| <a href="#">ADR-003</a> | Highscore als View                      | UC08, <code>vw_Highscore</code>         |
| <a href="#">ADR-004</a> | Cell-Uniqueness via Trigger             | UC04, V06                               |
| <a href="#">ADR-005</a> | AppUser statt User                      | T-SQL Reserved                          |
| <a href="#">ADR-006</a> | RowIdx/ColIdx statt Row/Col             | T-SQL Reserved                          |
| <a href="#">ADR-007</a> | Keine Solution-Spalte                   | UC04 strict                             |
| <a href="#">ADR-008</a> | Pencil Marks als eigene Tabelle         | UC14                                    |
| <a href="#">ADR-009</a> | Pause via TotalPausedSeconds            | UC13                                    |
| <a href="#">ADR-010</a> | Solver limit 2 Lösungen                 | UC11, V07                               |
| <a href="#">ADR-011</a> | Test-Pyramide                           | README §3.2                             |
| <a href="#">ADR-012</a> | Score-Formel                            | UC08                                    |
| <a href="#">ADR-013</a> | Hint-Strategie                          | UC07                                    |
| <a href="#">ADR-014</a> | 3 zusätzliche UCs                       | README §2.3                             |
| <a href="#">ADR-015</a> | Single-Image Docker (DB + App)          | <a href="#">§7.5</a>                    |

## Verweise

- [Kapitel 1 — Einführung und Ziele](#)
- [Kapitel 7 — Verteilungssicht](#)
- [Kapitel 8 — Querschnittliche Konzepte](#)
- **Use-Cases-Dokument**
- **Validation-Regeln**
- **ER-Modell**
- **Funktionalitäts-Matrix**
- **Datenbank-Skript**
- **Aufgabenstellung** — Aufgabenstellung (autoritativ)



# 10 Qualitätsanforderungen

arc42 v8.2 · Skills Battle 2026 — Application Development — Killer Sudoku Quelldokument  
(autoritativ): **Aufgabenstellung**

Dieses Kapitel präzisiert die in §1.2 genannten Top-Ziele in Form eines Quality-Trees und konkreter, testbarer Quality-Szenarien.

## §10.1 Quality-Tree

Der Tree gliedert das übergeordnete Ziel "Qualität" in fünf Attribute. Jedes Blatt ist über ein Szenario in §10.2 operationalisiert.



**Zuordnung Top-Ziele → Quality-Tree-Knoten:**

| Top-Ziel (siehe §1.2)      | Tree-Knoten                                                |
|----------------------------|------------------------------------------------------------|
| Q1 Korrektheit des Solvers | KO1, KO2, PE1                                              |
| Q2 Lösungs-Eindeutigkeit   | KO1 (Branch <code>Solutions ≥ 2</code> ), siehe <b>V07</b> |
| Q3 Test-Abdeckung          | WA1                                                        |

## §10.2 Quality-Szenarien

Jedes Szenario ist tabellarisch nach dem arc42-Schema **Quelle · Stimulus · Artefakt · Reaktion · Antwortmaß** beschrieben. Spalte "Test-Anker" verlinkt die geplanten Unit-/Integration-/E2E-Tests (siehe **Test-Protokoll**).

## Q01 — Solver-Korrektheit (Eindeutigkeit)

| Feld         | Wert                                                                                                                                                                  |
|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Quelle       | Wettbewerbs-Prüfer / interner Integration-Test                                                                                                                        |
| Stimulus     | Solver bekommt ein Puzzle mit genau einer Lösung übergeben                                                                                                            |
| Artefakt     | <code>ISolverService.Solve(givenValues, cages)</code>                                                                                                                 |
| Reaktion     | Solver gibt <code>SolveResult { Solutions = 1, Solution = ...}</code> zurück; bei mehrdeutigem Puzzle stoppt er bei 2 Lösungen und liefert <code>Solutions = 2</code> |
| Antwortmaß   | <b>100%</b> der Test-Cases (beide README-Beispiele + Edge-Cases unlösbar / mehrdeutig) liefern korrekten Solutions-Count                                              |
| README-Beleg | §1 "The solution <b>must be unique</b> ." + §2.1 UC11 "Solutions <b>must be calculated with an algorithm</b> ."                                                       |
| Test-Anker   | UC11 → AC11.1 (siehe use-cases.md UC11)                                                                                                                               |

## Q02 — Solver-Performance

| Feld         | Wert                                                                                                                              |
|--------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Quelle       | Wettbewerbs-Prüfer (Demo-Lauf)                                                                                                    |
| Stimulus     | Solver wird mit schwerem Puzzle (Difficulty 3) aufgerufen                                                                         |
| Artefakt     | <code>ISolverService.Solve</code> (Backtracking + Constraint-Propagation)                                                         |
| Reaktion     | Solver terminiert und liefert Ergebnis                                                                                            |
| Antwortmaß   | <b>≤ 2 Sekunden</b> auf lokaler Test-Maschine                                                                                     |
| README-Beleg | implizit aus UC05 ("solvability check vor Speichern") und UC07 ("ask for a hint") — bei längerer Laufzeit ist UX nicht akzeptabel |
| Test-Anker   | UC11 → AC11.2                                                                                                                     |

### Q03 — Login-Security (kein Username-Leak)

| Feld         | Wert                                                                                                                                                                       |
|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Quelle       | Externer Angreifer / Security-Reviewer                                                                                                                                     |
| Stimulus     | Login-POST mit unbekanntem Username oder falschem Passwort                                                                                                                 |
| Artefakt     | <code>IAuthService.LoginAsync</code>                                                                                                                                       |
| Reaktion     | Server antwortet mit <b>generischer</b> Fehlermeldung "Username oder Passwort falsch." — <b>keine</b> Unterscheidung zwischen "User existiert nicht" und "Passwort falsch" |
| Antwortmaß   | Response-Body, Statuscode und Antwortzeit identisch für beide Fälle (kein Timing-Leak via getrennten Code-Pfaden)                                                          |
| README-Beleg | §2.1 UC2 "a login <b>is required</b> ." in Kombination mit Sicherheits-Best-Practices ( <b>V03</b> )                                                                       |
| Test-Anker   | UC03 → AC03.1                                                                                                                                                              |

### Q04 — Page-Load (Play-Screen)

| Feld         | Wert                                                                                            |
|--------------|-------------------------------------------------------------------------------------------------|
| Quelle       | End-Nutzer                                                                                      |
| Stimulus     | Klick auf Puzzle in Liste S4 → Navigation zu Screen S6 ( <code>/puzzles/{id}/play</code> )      |
| Artefakt     | Blazor-Server-Page <code>PlayPuzzle.razor</code> + initiale SignalR-Verbindung + DB-Query Cages |
| Reaktion     | Grid + Cages + Toolbar werden vollständig gerendert                                             |
| Antwortmaß   | <b>&lt; 1 Sekunde</b> bei MS-SQL Server lokal (Loopback) und Cold-Cache                         |
| README-Beleg | impliziert durch §1.1 "appropriate navigation" + "controls for all functions"                   |
| Test-Anker   | Manueller UI-Test im <b>Test-Protokoll</b>                                                      |

## Q05 — Auth-Cookie-Sicherheit

| Feld         | Wert                                                                      |
|--------------|---------------------------------------------------------------------------|
| Quelle       | Security-Reviewer / Browser DevTools                                      |
| Stimulus     | Erfolgreicher Login (UC03)                                                |
| Artefakt     | ASP.NET Core Authentication Cookie                                        |
| Reaktion     | Cookie wird mit Flags <b>HttpOnly + Secure + SameSite=Lax</b> ausgestellt |
| Antwortmaß   | Set-Cookie-Header enthält alle drei Attribute (binary check: pass/fail)   |
| README-Beleg | §2.1 UC2 "login is required" + Sicherheits-Defaults nach <b>V15</b>       |
| Test-Anker   | UC03 → AC03.3                                                             |

## Q06 — Test-Coverage

| Feld         | Wert                                                                                                                                                                                                                   |
|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Quelle       | Wettbewerbs-Prüfer (Submission-Bewertung)                                                                                                                                                                              |
| Stimulus     | Build + Test-Lauf via <code>dotnet test</code>                                                                                                                                                                         |
| Artefakt     | xUnit Test-Suite + Coverlet Coverage-Report                                                                                                                                                                            |
| Reaktion     | Report zeigt Coverage-Wert pro Projekt                                                                                                                                                                                 |
| Antwortmaß   | <b>≥ 80%</b> Line- + Branch-Coverage über Solver / Service-Layer                                                                                                                                                       |
| README-Beleg | §3.1 "For each use case, create at least one positive, one negative, and, where possible, one test case for boundary conditions." + §3.2 "Running the test cases <b>should be implemented with a test framework.</b> " |
| Test-Anker   | Coverlet-Report als Submission-Artefakt; siehe Top-Ziel <b>Q3</b> in §1.2                                                                                                                                              |

## §10.3 Querverweise

- [Top-Ziele §1.2](#) — drei priorisierte Ziele
- Validation V01–V16 — Detailregeln pro Layer (Client / Server / DB)
- Use Cases UC01–UC14 — Acceptance Criteria (AC) je UC
- **Test-Protokoll** — konkrete Test-Cases mit Erwartung
- [Kapitel 11 — Risiken](#) — Risiken, die die Qualitätsziele bedrohen

# 11 Risiken und technische Schulden

arc42 v8.2 · Skills Battle 2026 — Application Development — Killer Sudoku Quelldokument  
(autoritativ): **Aufgabenstellung**

Dieses Kapitel listet Wettkampf-spezifische Risiken (R01–R08) sowie bewusste technische Schulden (TD01–TD03) zum Stand der Architektur-Festschreibung. Bewertungs-Skalen:

- **Wahrscheinlichkeit (W):** niedrig / mittel / hoch
- **Auswirkung (A):** niedrig / mittel / hoch / kritisch (kritisch = führt direkt zu UC-/AC-Verletzung)

## §11.1 Wettkampf-Risiken

### R01 — Solver-Performance bei worst-case-Cage-Konfigurationen

| Attribut           | Wert                                                                                                                                |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Beschreibung       | Backtracking-Solver kann bei dünn besetzten oder ungünstig zerlegten Cage-Layouts in eine kombinatorische Explosion laufen.         |
| Wahrscheinlichkeit | mittel                                                                                                                              |
| Auswirkung         | hoch — Verletzung von Quality-Szenario <b>Q02</b> ( $\leq 2$ s)                                                                     |
| Mitigation         | Constraint-Propagation pro Cage (Min-/Max-Sum-Bounds), Variable-Ordering MRV (Most-Restricted-Variable), harter Timeout im Service. |
| Test-Anker         | UC11 → AC11.2 (siehe use-cases.md UC11)                                                                                             |

### R02 — Multi-Solution-Erkennung übersehen (Pattern 2 "Spec-Detail-Blindheit")

| Attribut           | Wert                                                                                                                                                                                                                |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Beschreibung       | Naïve Solver-Implementierung gibt die erste gefundene Lösung zurück und stoppt — verletzt das README-Strict-Wort "The solution <b>must be unique.</b> " (§1). Multi-Solution-Puzzles würden fälschlich gespeichert. |
| Wahrscheinlichkeit | hoch (klassischer Fehler bei Killer-Sudoku-Implementierungen)                                                                                                                                                       |
| Auswirkung         | kritisch — Verletzung von <b>V07</b> und Top-Ziel <b>Q2</b>                                                                                                                                                         |
| Mitigation         | Solver sucht aktiv nach <b>2.</b> Lösung und bricht erst dann ab;<br><code>SolveResult.Solutions ∈ {0, 1, 2}</code> ; UC05 speichert ausschliesslich bei <code>Solutions == 1</code> .                              |
| Test-Anker         | UC05 → AC05.2; UC11 → AC11.1                                                                                                                                                                                        |

## R03 — SQL-Constraint-Lücken (Cage-Trigger fehleranfällig)

| Attribut           | Wert                                                                                                                                                                                                                                                                                                                    |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Beschreibung       | Der INSTEAD-/AFTER-Trigger <code>trg_CageCell_UniquePerPuzzle</code> (siehe <b>Datenbank-Skript</b> ab Zeile 99) ist die einzige DB-seitige Garantie, dass eine Zelle nicht in zwei Cages desselben Puzzles liegt. Fehler im Trigger oder Batch-Inserts mit deaktiviertem Trigger könnten inkonsistente Daten zulassen. |
| Wahrscheinlichkeit | niedrig                                                                                                                                                                                                                                                                                                                 |
| Auswirkung         | hoch — Datenintegritäts-Verletzung gegenüber <b>V06</b>                                                                                                                                                                                                                                                                 |
| Mitigation         | Defense in Depth — Application-Layer-Validierung in <code>IPuzzleService.ValidateStructureAsync</code> zusätzlich zur Trigger-Prüfung; Integration-Test mit absichtlich kollidierendem Insert.                                                                                                                          |
| Test-Anker         | UC04 → AC04.2; Integration-Test "Doppelte Cage-Cell wird vom Trigger geblockt"                                                                                                                                                                                                                                          |

## R04 — Blazor-Server-Circuit-Disconnects bei langen Pausen (UC13)

| Attribut           | Wert                                                                                                                                                                                                                                                                                  |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Beschreibung       | Blazor-Server-Apps halten pro User eine SignalR-Circuit (siehe <b>Funktionalitäts-Matrix</b> "SignalR-Verbindung permanent"). Bei Pause + Tab-Wechsel kann die Circuit getrennt werden — laufende Game-State-Manipulationen würden verloren gehen, falls UC13-Auto-Save nicht greift. |
| Wahrscheinlichkeit | mittel                                                                                                                                                                                                                                                                                |
| Auswirkung         | mittel — UC13 AC13.2 (Resume restored alle Zellen) bedroht                                                                                                                                                                                                                            |
| Mitigation         | Persistenz pro Move (UC13 "Browser-Tab schliessen ohne Pause → State wird trotzdem persistiert (auto-save bei jedem Move)"), serverseitiges Resume aus DB statt aus Circuit-State.                                                                                                    |
| Test-Anker         | UC13 → AC13.1, AC13.2                                                                                                                                                                                                                                                                 |

## R05 — Test-Coverage-Schwächen bei Solver-Edge-Cases

| Attribut           | Wert                                                                                                                                                                                                                                             |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Beschreibung       | Solver hat viele Code-Pfade (Cage mit 1 Zelle, Cage über ganze Zeile, unlösbares Puzzle, mehrdeutiges Puzzle, leeres Grid). Ohne gezielten Edge-Case-Plan kann die 80%-Marke (Quality-Szenario <a href="#">Q06</a> ) am Solver allein scheitern. |
| Wahrscheinlichkeit | mittel                                                                                                                                                                                                                                           |
| Auswirkung         | hoch — verletzt Top-Ziel <b>Q3</b> und damit Submissions-Qualität                                                                                                                                                                                |
| Mitigation         | Solver-Unit-Tests systematisch nach der Edge-Case-Liste in <code>functionality.md</code> "Kritische Abhängigkeiten" (5+ Tests: README-Beispiele, unlösbar, mehrdeutig, 1-Zell-Cage, voller Zeilen-Cage).                                         |
| Test-Anker         | UC11 — separate Test-Klasse <code>SolverServiceTests</code>                                                                                                                                                                                      |

## R06 — Sum-Check Pre-Validation übersehen (Pattern 5 "Improvisierter Format-String")

| Attribut           | Wert                                                                                                                                                                                                                                                                                 |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Beschreibung       | README §2.3 verlangt explizit: "Use the value [Total = 405] for a 'simple' validation <b>before</b> checking the solution with an algorithm." Eine Implementierung, die direkt den vollen Algorithmus aufruft, ignoriert die README-Vorgabe — auch wenn das Endergebnis korrekt ist. |
| Wahrscheinlichkeit | mittel                                                                                                                                                                                                                                                                               |
| Auswirkung         | mittel — Verletzung einer explizit benannten Spec-Anforderung                                                                                                                                                                                                                        |
| Mitigation         | <code>IGameService.CheckSolutionAsync</code> ruft als ersten Schritt den Sum-Check (siehe <b>V09</b> ) auf; Unit-Test verifiziert, dass bei <code>Summe ≠ 405</code> der Solver nicht aufgerufen wird.                                                                               |
| Test-Anker         | UC09 → AC09.1                                                                                                                                                                                                                                                                        |

## R07 — Spec-Verletzung "No solution is recorded" durch versehentliches Speichern

| Attribut           | Wert                                                                                                                                                                                                                           |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Beschreibung       | README §2.1 UC4: "No solution is recorded." Wenn z.B. zu Debug-Zwecken oder per ORM-Auto-Migration eine Spalte <code>Solution</code> in <code>Puzzle</code> landet, wird die Spec verletzt — auch wenn die Spalte leer bleibt. |
| Wahrscheinlichkeit | niedrig                                                                                                                                                                                                                        |
| Auswirkung         | hoch — direkte Verletzung eines Strict-Wortes der Spec                                                                                                                                                                         |
| Mitigation         | Schema in <b>Datenbank-Skript</b> ohne Solution-Spalte; Integration-Test gegen <code>INFORMATION_SCHEMA.COLUMNS</code> stellt sicher, dass keine <code>Solution</code> -Spalte existiert (siehe <b>ER-Modell</b> ).            |
| Test-Anker         | UC04 → AC04.4                                                                                                                                                                                                                  |

## R08 — Reserved-Keyword-Konflikte in T-SQL (User, Row, Col)

| Attribut           | Wert                                                                                                                                                                                                                                                           |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Beschreibung       | Die in der Domäne natürlichen Namen <code>User</code> , <code>Row</code> , <code>Col</code> sind in T-SQL reservierte Keywords. Ohne Anpassung schlagen DDL-/DML-Statements fehl oder benötigen Bracket-Quoting <code>[User]</code> .                          |
| Wahrscheinlichkeit | hoch (würde ohne Umbenennung sicher auftreten)                                                                                                                                                                                                                 |
| Auswirkung         | mittel (Build-Fail, leicht zu beheben — aber unschön und Quelle für Folgebugs)                                                                                                                                                                                 |
| Mitigation         | Umbenennung gemäss erm.md Design-Entscheidungen: Tabelle <code>AppUser</code> statt <code>User</code> , Spalten <code>RowIdx</code> / <code>ColIdx</code> statt <code>Row</code> / <code>Col</code> . Bereits konsistent in <b>Datenbank-Skript</b> umgesetzt. |
| Test-Anker         | Build-/Migrations-Lauf des <b>Datenbank-Skript</b> -Scripts                                                                                                                                                                                                    |

## §11.2 Technische Schulden (Stand 2026-05-27, vor App-Implementierung)

Die folgenden Schulden sind **bewusste Entscheidungen** für den Wettkampf-Scope. Jede Schuld nennt den Grund und die Konsequenz für einen späteren Produktiv-Einsatz.



## TD01 — Keine EF Migrations, nur Raw SQL

| Attribut                | Wert                                                                                                                                                                                           |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Beschreibung            | DB-Schema wird über das einzelne Skript <b>Datenbank-Skript</b> gepflegt — kein <code>dotnet ef migrations add ...</code> .                                                                    |
| Begründung              | README §1.3 verlangt explizit: "Create for each table a SQL statement and save them in a script called sudoku.sql." Eine EF-Migration würde paralleles, alternatives Wahrheit-Schema erzeugen. |
| Folge bei Produktiv-Use | Schema-Evolution erfordert manuelle SQL-Patches; kein Rollback-Mechanismus; keine automatisierte Schema-Drift-Erkennung.                                                                       |

## TD02 — Keine i18n-Vorbereitung (UI nur Deutsch)

| Attribut                | Wert                                                                                                                                                                                        |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Beschreibung            | UI-Texte (Fehlermeldungen, Labels, Validierungs-Strings) sind direkt in Razor-Components hardcoded in <b>Deutsch</b> . Kein <code>LocalizedString</code> , keine <code>.resx</code> -Files. |
| Begründung              | Wettkampf-Scope; README definiert keine Mehrsprachigkeit; reduziert Aufwand zugunsten Test-Coverage.                                                                                        |
| Folge bei Produktiv-Use | Vollständiges Refactoring zu Localizer notwendig für Englisch/Französisch; Test-Updates für lokalisierte Strings.                                                                           |

## TD03 — Keine Multi-User-Concurrency-Tests (Blazor-Server-Single-Circuit-Annahme)

| Attribut                | Wert                                                                                                                                                                                                                                                     |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Beschreibung            | Tests gehen davon aus, dass pro User maximal <b>eine</b> aktive Blazor-Circuit existiert. Race-Conditions bei parallelen Tabs / mehreren Devices desselben Users werden nicht durch Tests abgedeckt.                                                     |
| Begründung              | Wettkampf-Scope. UC13 AC13.3 fordert lediglich "Nur 1 aktives Game pro User-Puzzle-Kombi gleichzeitig" — das ist über den DB-Filtered-Index <code>UX_Game_ActiveOnly</code> (siehe <b>Datenbank-Skript</b> Zeile 155) abgesichert, nicht über App-Tests. |
| Folge bei Produktiv-Use | Concurrency-/Last-Tests erforderlich; ggf. Optimistic-Concurrency-Token auf <code>Game</code> (rowversion).                                                                                                                                              |

## §11.3 Querverweise

- [§10.2 Quality-Szenarien](#) — die durch Risiken bedrohten Antwortmasse

- Validation V01–V16
- Use Cases mit Acceptance Criteria
- ER-Modell und Design-Entscheidungen
- DB-Schema

# 12 Glossar

---

arc42 v8.2 · Skills Battle 2026 — Application Development — Killer Sudoku Quelldokument  
(autoritativ): **Aufgabenstellung**

Alle fachlichen und technischen Begriffe der Architektur-Doku, alphabetisch sortiert.  
Quellenangaben verweisen auf das Wettbewerbs-README (**Aufgabenstellung**) bzw. die  
ergänzenden Quell-Dokumente unter **docs/**.

---

| Term                             | Definition · Quelle                                                                                                                                                                            |
|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>AC (Acceptance Criterion)</b> | Testbares Kriterium pro Use Case; mappt 1:1 auf einen Test-Case im Test-Protokoll. Siehe Notation in <b>Use-Cases-Dokument</b> .                                                               |
| <b>AppUser</b>                   | Benutzer-Entität in der DB. Namens-Workaround, weil <code>User</code> ein reserviertes T-SQL-Key-Word ist (siehe <a href="#">R08</a> ). Tabellen-Definition: <b>Datenbank-Skript</b> Zeile 36. |
| <b>arc42</b>                     | Architektur-Dokumentations-Template (arc42.org), v8.2, 12 Kapitel. Siehe Index <a href="#">00-README.md</a> .                                                                                  |
| <b>Auto-Solver</b>               | Algorithmus aus UC11, der ein Puzzle automatisch löst. Service-Interface <code>ISolverService</code> . Siehe use-cases.md UC11.                                                                |
| <b>Backtracking</b>              | Solver-Strategie: rekursive Tiefensuche mit Constraint-Checks, Rücksprung bei Konflikt. Eingesetzt im <code>SolverService</code> für Killer-Sudoku (UC11).                                     |
| <b>bUnit</b>                     | .NET Testing Library für Blazor-Component-Tests. Verwendet für Component-Level-Tests von <code>PuzzleGrid.razor</code> etc. (siehe <b>Funktionalitäts-Matrix</b> ).                            |
| <b>Cage</b>                      | Zell-Gruppe mit vorgegebener Summe (Killer-Sudoku-spezifisch). README §1: "The sum of all numbers in a cage must match the small number printed in its corner."                                |
| <b>Cage-Sum</b>                  | Soll-Summe eines Cages, Wertebereich 1–45 (1 Zelle min 1, 9 Zellen max $1+...+9 = 45$ ). DB-Constraint <code>CK_Cage_Sum_Range</code> in <b>Datenbank-Skript</b> .                             |
| <b>Cell / Zelle</b>              | Ein Feld im 9×9-Grid; identifiziert durch <code>(RowIndex, ColIdx)</code> mit Werten 0–8. Wert einer Spielzelle: 1–9 oder leer.                                                                |
| <b>Difficulty</b>                | Schwierigkeitsgrad 1, 2 oder 3. README §2.1 UC4: "the difficulty level from 1-3". DB-Constraint <code>CK_Puzzle_Difficulty</code> .                                                            |
| <b>Game</b>                      | Eine Spielsession eines Users an einem Puzzle. Tabellen-Definition: <b>Datenbank-Skript</b> Zeile 128.                                                                                         |
| <b>Hint</b>                      | Algorithmischer Tipp; UC07. Strategie: Naked-Single → Cage-Forced → Solver-Fallback (siehe use-cases.md UC07).                                                                                 |
| <b>Highscore</b>                 | Score-Ranking gemäss Formel UC08; persistiert als View <code>vw_Highscore</code> (siehe <b>Datenbank-Skript</b> Zeile 215).                                                                    |
| <b>HintsUsed</b>                 | Anzahl der in einem Game verwendeten Hints; score-relevant gemäss UC08-Formel. README §2.1 UC8: "based on the time needed and the number of hints used".                                       |
| <b>Killer Sudoku</b>             | Sudoku-Variante mit zusätzlichen Cage-Summen. README §1: "various tricky variants. One variant is to fill in the squares based on a set of sums over several squares."                         |
| <b>Nonet</b>                     | Ein 3×3-Block im Sudoku-Grid; insgesamt 9 Nonets. README §1: "Each row, column, and <b>nonet</b> contains each number exactly once".                                                           |
| <b>Pencil Mark</b>               | Kandidaten-Notiz pro Zelle (kleine Markierungen 1–9 als mögliche Werte). UC14, eigene Tabelle <code>PencilMark</code> .                                                                        |

|                      |                                                                                                                                                                                                                                                                                  |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Puzzle</b>        | Eine Killer-Sudoku-Aufgabe (Cages + Difficulty). README §2.1 UC4: "No solution is recorded" — Lösung wird <b>nicht</b> persistiert.                                                                                                                                              |
| <b>Score</b>         | Punkte gemäss Formel <code>max(0, 10000 - TimeSeconds - HintsUsed × 300)</code> (siehe use-cases.md UC08).                                                                                                                                                                       |
| <b>SignalR</b>       | Real-Time-Kommunikations-Framework für Blazor-Server-Circuits. Permanente Verbindung Server ↔ Client (siehe functionality.md "Stack-Konventionen").                                                                                                                              |
| <b>Solvability</b>   | Existenz mindestens einer Lösung. UC05-Bedingung: "The puzzle can only be saved if it is solvable." (README §2.1 UC5).                                                                                                                                                           |
| <b>Solver</b>        | Domänen-Service <code>ISolverService</code> (UC11). Interface-Definition: functionality.md "Service-Interfaces".                                                                                                                                                                 |
| <b>Sum-Check 405</b> | Schnell-Validierung: Summe aller 81 Zellen-Werte = $9 \times 45 = 405$ . README §2.3: "Calculate the sum of all numbers in the completed Sudoku. The value can be determined unambiguously." Eingesetzt <b>vor</b> der vollständigen Algorithmus-Validierung. Siehe <b>V09</b> . |
| <b>TDD</b>           | Test-Driven Development; Workflow Red → Green → Refactor. README §3.1: "Plan your test cases <b>before</b> you start the implementation."                                                                                                                                        |
| <b>TimeSeconds</b>   | Spieldauer in Sekunden, abzüglich Pausen: <code>TimeSeconds = DATEDIFF(SECOND, StartTime, EndTime) - TotalPausedSeconds</code> . Score-relevant (UC08). Siehe <b>UC10</b> und <b>UC13</b> .                                                                                      |
| <b>Uniqueness</b>    | Lösung muss eindeutig sein. README §1: "The solution <b>must be unique</b> ." Strict-Wort; Multi-Solution-Puzzles werden abgelehnt (siehe <a href="#">R02</a> , <b>V07</b> ).                                                                                                    |
| <b>Use Case</b>      | Anwendungsfall, Notation siehe <b>Use-Cases-Dokument</b> . UC01–UC11 aus README §2.1, UC12–UC14 selbst gewählt gemäss README §2.3.                                                                                                                                               |
| <b>V01–V16</b>       | Validation-Regeln aus <b>Validation-Regeln</b> . Jede Regel ist Layer-zugeordnet (Client / Server / DB).                                                                                                                                                                         |
| <b>xUnit</b>         | .NET Unit-Testing-Framework, Standard-Wahl für Service-/Solver-Tests (siehe functionality.md "Kritische Abhängigkeiten").                                                                                                                                                        |